

تصميم منطقي

تلخيص شامل (نظري)

حسن إسماعيل صالح
الجامعة الإسلامية تخصص هندسة ذكاء إصطناعي
التاريخ :

بسم الله الرحمن الرحيم

الحمد لله رب العالمين، والصلاة والسلام على سيدنا محمد وعلى آله وصحبه أجمعين.

يسرني أن أضع بين أيديكم هذا الملخص لمادة تصميم منطقي 1 ، والذي قمت بإعداده خلال دراستي للمساق بهدف جمع أهم الأفكار والمفاهيم والقوانين الواردة في المادة بصورة مرتبة ومبسطة تساعد على فهمها ومراجعتها بشكل أسرع. وقد حاولت في هذا العمل تلخيص المحتوى الأساسي للمادة مع التركيز على النقاط المهمة التي يحتاجها الطالب أثناء الدراسة أو قبل الامتحانات، وذلك لتسهيل الطريق على الطلبة الذين سيدرسون هذا المساق في المستقبل بإذن الله.

كما أتقدم بجزيل الشكر والتقدير إلى الأستاذ الدكتور الفاضل وسام عاشور، والمهندس القدير هاشم حجازي، اللذين ألهما بصيرتنا بجهودهما الحثيثة في شرح المادة وتوضيح مفاهيمها، ولأسلوبهما المميز في التدريس والتوجيه الذي كان له أبلغ الأثر في عمق فهمنا للمادة وتحقيق أقصى استفادة منها.

هذا الملخص ما هو إلا جهد بشري قابل للخطأ أو النقص، وقد كتبته بنية الإفادة ونشر العلم بين الطلبة. أسأل الله أن ينفع به، وأن يجعله عملاً خالصاً لوجهه الكريم. كما أرجو من كل من ينتفع به أن يذكر شهادتنا وجرحانا بدعوة صادقة .

والحمد لله رب العالمين.



Chapter 1 : Digital Systems and Binary Numbers

Lecture 1

- Decimal (عشري) : The one that we use in our real life that consists of digits $(0, 1, 2, \dots, 9)$, we denote an integer on decimal system by $(x)_{10}$.
- Binary (ثنائي) : The one that we use for electric devices such as computer , devices understand the language of (On/Off) , it consists of digits $(0, 1)$ only and we denote the integer on binary system by $(x)_2$
- Octal (ثماني) : The main reason for it's existence is optimizing the efficiency of reading a piece of information that's represented in binary by compressing each 3 digits into single block , it consists of digits $(0, 1, 2, \dots, 7)$ and we denote it by $(x)_8$.
- Hexadecimal (سادس عشر) : To avoid the confusion of the representation were we treat higher values than 9 as single digits or a combination , we introduce letters A, B, C, D, E, F to denote the range 10-15 respectively , similarly to Octal system , we optimize the efficiency by compressing each 4 digits into single block , we denote it by $(x)_{16}$.

Observation: The set of digits we can use for the k_{th} numerical system is $0, 1, 2, 3, \dots, k - 1$. thus for each placement at $(x)_k$ there'll be no placement $x_i \geq k$.

The Octal and Hexadecimal systems are **Mediator link** between Decimal and Binary systems.

Bytes and Computer Capacity

Computer capacity is usually given in bytes, where a byte is equal to eight bits and can accommodate the code of one keyboard character. In computer work, capacity prefixes represent powers of 2: 2^{10} is referred to as **K (kilo)**, 2^{20} as **M (mega)**, 2^{30} as **G (giga)**, and 2^{40} as **T (tera)**.

Example: $4K = 2^{12} = 4,096$ bytes, and a computer hard disk with four gigabytes of storage has $4G = 2^{32}$ bytes.

Converting from Binary, Octal and Hex to Decimal

- To Convert a number of base b with x decimal places in the integer part and y decimal places in the fractional part , Formally : $x = (\underbrace{d_1 d_2 \dots d_x}_{\text{integer part}} . \underbrace{d_{x+1} d_{x+2} \dots d_{x+y}}_{\text{fractional part}})_b$ to a decimal , we follow

the following

1. Define weights (powers of b) as following

$$x = (\underbrace{d_1^{x-1} d_2^{x-2} \dots d_{x-2}^2 d_{x-1}^1 d_x^0}_{\text{integer part}} . \underbrace{d_{x+1}^{-1} d_{x+2}^{-2} \dots d_{x+y}^{-y}}_{\text{decimal part}})_b$$

For example : $x = (d_1^2 d_2^1 d_1^0 . d_4^{-1} d_5^{-2} d_6^{-3})_2$ to decimal is $\sum_{k=-3}^2 d_k \cdot 2^{w_k}$

2. Compute the sum of contributions by weights :

$$\sum_{k=1}^{x+y} d_k \cdot b^{w_k}$$

Example. Evaluate $(101.101)_2$: For integer part from left to right : $1 \times 2^0 + 0 \cdot 2^1 + 1 \times 2^2 = 1 + 4 = 5$

, now for the fractional part $= 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} = \frac{1}{2} + \frac{1}{8} = \frac{5}{8} = 0.625$ thus the binary number represents the decimal number 5.625.

Example. Evaluate $(26.71)_8 = (6 \times 1 + 2 \times 8^1 + 7 \times 8^{-1} + 1 \times 8^{-2}) = 22 + \frac{7}{8} + \frac{1}{64}$.

Example. Evaluate $(1A2.B1)_{16} = 2 \times 1 + 10 \times 16^1 + 1 \times 16^2 + \frac{11}{16} + \frac{1}{256} = 418.6914$

Converting from Decimal to Binary, Octal and Hex

To convert a number n that's represented in base 10 i.e. decimal to another system with base b we follow the following process : (Phase 1 : for integer part) :

1. Perform repeated division by b . Let n_0 be the integer n .

2. For $k = 1, 2, \dots$ until $n_{k-1} = 0$:

$$d_k = n_{k-1} \bmod b$$

$$n_k = \lfloor n_{k-1}/b \rfloor$$

3. The representation is the sequence of remainders in reverse: $(d_m d_{m-1} \dots d_1)_b$.

Phase 2 : for the decimal part

1. Let f_0 be the fractional part $\{n\}$.

2. For $k = 1, 2, \dots, 6$:

$$d_{-k} = \lfloor f_{k-1} \times b \rfloor$$

$$f_k = \{f_{k-1} \times b\}$$

3. The representation is the sequence $(d_{-1} d_{-2} \dots d_{-6})_b$.

And finally the answer is the result of phase 1 + ' ' (separation) + phase 2 result (with trailing zeros omitted).

Remark. The more k we process after the fixed point " ", the higher precision we get.

Example. $(25.2)_{10} \rightarrow (x)_8$.

We get integer part first :

$$25 \xrightarrow{r=1} 3 \xrightarrow{r=3} 0$$

this means the integer part is $(31)_8$.

We get fractional part

$$0.2 \xrightarrow{1} 0.6 \xrightarrow{4} 0.8 \xrightarrow{6} 0.4 \xrightarrow{3} 0.2 \xrightarrow{1} 0.6 \xrightarrow{4} 0.8 \xrightarrow{6} 0.4 \xrightarrow{3} 0.2$$

by taking it in order we get : 146314 thus the answer is

$$(31.146314)_8$$

Conversion from base x to base y

- General Conversion between bases : We can convert from specific base to another by knowing how to transform in/out of specific base , We already learned about decimals (base 10) thus we can have two phases :

1. Transform the given integer in base x to base 10

2. Transform the result in base 10 to base y .

- Binary (2) to Octal (8) : we convert bit-by-bit , each triplet of bits (3 consecutive bits) correspond to an octal digit which is the binary-to-decimal evaluation of that triplet , In case we lack a digit or two we assume 0's , As usual we process integer part from right to left and the fractional part from left to right.

Example : Convert $(11101011.10110110)_2$ to Octal.

011
101
011
.
101
101
100

now we convert each of these blocks to decimal

$$(353.554)_8$$

Remark : Octal to Bin conversion can be done by reverse-engineering i.e. via expanding instead of composing.

Example : Convert $(611.116)_8$ to binary.

110
001
001
.
001
001
110

- Binary (2) to Hexadecimal (16) (and viceversa) can be done with the same idea of binary-octal by extending the idea to blocks of size 4 instead of 3.
- Octal (8) to Hexadecimal (16) (and viceversa) can be done by converting the given integer into binary and use the idea of blocks splitting in order to convert in a fast way.

Addition and Multiplication

The addition and multiplication operations on all systems follows the carry principle as following for addition:

- let's have two integer x and y and base b
- we start from right to left.
- we add the current digits x_i and y_i .
- We put $(x_i + y_i) \bmod b$ at the current digit's result and we give a carry of $\left\lfloor \frac{x_i + y_i}{b} \right\rfloor$ to next place and continue the operation.
- If in if the last place has also a carry then we've a new place.

For multiplication , we do the same algorithm of multiplying on base 10 integers (convolve digits and shift a place for each digit) , We'll take multiple examples to get this cleaner
$$\begin{array}{r} (26)_7 \\ + (15)_7 \\ \hline (44)_7 \end{array}$$
 Here we've $6 + 5 = 11$ which has remainder of 4 modulus 7 , and we've only one seven on 11.

another example
$$\begin{array}{r} (1)1 \\ 0 (22)_3 \\ + 0(21)_3 \\ \hline 120 \end{array}$$
 , we can also extend this to Hex system :
$$\begin{array}{r} (1) (1) \\ 0 \ 5 \ F_{16} \\ + \ 0 \ B \ E_{16} \\ \hline 1 \ 1 \ D_{16} \end{array}$$

An example for multiplication :

$$\begin{array}{r}
 \begin{array}{c} (2) \\ 2 \text{ B}_{16} \end{array} \\
 \times \quad 1 \text{ 3}_{16} \\
 \hline
 8 \quad 1 \\
 + \quad 2 \text{ B} \quad 0 \\
 \hline
 3 \text{ 3 } 1_{16}
 \end{array}$$

Binary Division:

Binary division follows the exact same mechanical process as decimal long division.

- Compare the divisor to the leftmost bits of the dividend.
- If the divisor fits (is less than or equal to the target bits), write a 1 in the quotient, subtract the divisor, and bring down the next bit.
- If it does not fit, write a 0 in the quotient and bring down the next bit.

Example: Divide 111011_2 by 101_2 .

1. $111 \div 101$: Fits 1 time. $111 - 101 = 10$. Bring down 0 $\rightarrow 100$.
2. $100 \div 101$: Doesn't fit. Write 0. Bring down 1 $\rightarrow 1001$.
3. $1001 \div 101$: Fits 1 time. $1001 - 101 = 100$. Bring down 1 $\rightarrow 1001$.
4. $1001 \div 101$: Fits 1 time. $1001 - 101 = 100$.

Result: Quotient = 1011_2 , Remainder = 100_2 .

Solving for an Unknown Base (Radix)

To solve algebraic equations where the base r is unknown, expand all numbers into polynomials in base 10 using the definition of positional number systems.

Example 1 (Basic): Determine the base r for the equation $14_r/2_r = 5_r$.

Expand into base 10: $(1 \cdot r^1 + 4 \cdot r^0)/2 = 5 \cdot r^0 \implies \frac{r+4}{2} = 5 \implies r+4 = 10 \implies r = 6$.

Example 2 (Quadratic Equivalent): The solutions to the equation $x^2 - 11_r x + 22_r = 0$ are $x = 3$ and $x = 6$. What is the base r ?

If $x = 3$ and $x = 6$ are solutions, the equation in base 10 must be: $(x-3)(x-6) = x^2 - 9_{10}x + 18_{10} = 0$.

Equate the coefficients from the base r equation to the base 10 equation:

Coefficient of x : $11_r = 9_{10} \implies 1 \cdot r^1 + 1 \cdot r^0 = 9 \implies r + 1 = 9 \implies r = 8$.

Constant term (verify): $22_r = 18_{10} \implies 2 \cdot r^1 + 2 \cdot r^0 = 18 \implies 2r + 2 = 18 \implies r = 8$.

Lecture 2

In this lecture , we'll discuss Complements المكملات , For each number written in base b there are b_{th} complement (i.e. radix complement) and $(b-1)_{th}$ complement (i.e. radix diminished complement).

The $(b-1)_{th}$ complement of $N = (d_{n-1}d_{n-2}...d_2d_1d_0)_b$ is $((b-1) - d_{n-1})((b-1) - d_{n-2})...((b-1) - d_2)((b-1) - d_1)((b-1) - d_0)$ which is equal to $\boxed{(b^n - 1) - N}$

Example. Find diminished complement for $(26)_7$: $((6-2)((6-6)))_7 = (40)_7$

Result. The diminished complementary of a number in base 2 is flipping 0's to 1's and 1's to 0's.

The $(b)_{th}$ complement of N is $(b^n) - N$ i.e. diminished complement +1 to last digit.

Example. $(28)_{10} \rightarrow (71)_{10} \rightarrow (72)_{10}$.

Another Example

$$\begin{array}{r} (23600)_7 \\ \rightarrow (43100) \\ 6's : (43066)_7 \\ + \quad 001 \\ \hline 7's : 43100_7 \end{array}$$

Benefits of Complementary :

- Simplifying Arithmetic Operations.
- Representing Negative Signed numbers.
- Simplifying Logic Operations.

Simplifying Arithmetic Operations

To Evaluate $(x)_b - (y)_b$ we evaluate the radix b_{th} complement of y i.e. $\overline{y}$ and evaluate $x + \overline{y}$ with **Discarding the end carry**.

Example 1. $5200 - 1450 = (5200) + (8550) = \boxed{1}3750$ here 1 is a carry so we ignore it thus answer is 3750.

Example 2. $(11010)_2 - (01011)_2 = (11010)_2 + (10101)_2 = (101111)_2 \xrightarrow{\text{discard carry}} (01111)_2 \Leftrightarrow 26 - 11 = 15$

Trap !!! : When you use binary system , you must add +1 after flipping because by flipping we're doing diminished radix complement and we're dealing with radix complement.

Representing Negative Signed Numbers

There are three different methods to represent negative numbers :

- Sign & Magnitude (Disadvantages : Complex Arithmetic Process and there are two signed zeros : +0 and -0).
- 1's Complement (Disadvantage : there are two signed zeros : +0 and -0).
- 2's Complement (Best Method).

How to represent each ?

- Sign & Magnitude : leftmost digit represents the sign (negative if 1) , and the rest or number representation is the magnitude.
- 1's Complement : Write binary representation and flip 0's to 1's and 1's to 0's.
- 2's Complement : find the 1's complement and add 1 to the rightmost bit (LSB). Note that carries may propagate to the left.

To Evaluate arithmetic , we proceed normally as we've discussed earlier with finding the 2's complement to the answer if negative.

★ **Visual : the three representations side by side (4-bit, value -5):**

Sign & Mag.

1	1	0	1
---	---	---	---

sign magnitude = 5

1's complement

1	0	1	0
---	---	---	---

flip every bit of +5

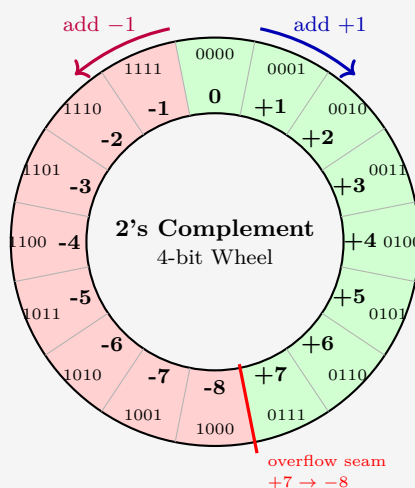
2's complement

1	0	1	1
---	---	---	---

1's compl. +1

(reference: +5 = 0101 in 4-bit binary)

★ **The 2's Complement Number Wheel (4-bit):** A circular layout where adding +1 walks clockwise and adding -1 walks counterclockwise. Overflow happens at the seam between +7 and -8.



Reading the wheel: positive values (green half) increase clockwise from 0000; negative values (red half) decrease counterclockwise from 1111 = -1. Adding +1 moves you one slot clockwise; adding -1 moves one slot counterclockwise. Going past +7 wraps to -8 — this is integer overflow. The 2's complement of any value is the slot diametrically opposite (e.g., +5 at 0101 is across from -5 at 1011).

Signed and Unsigned Numbers

★ An **overflow** occurs if you start with two n -bit numbers and their resulting sum occupies $n + 1$ bits. While this is easily fixed on paper by adding another bit, **overflow** is a problem in computers because the number of bits is finite. A result that exceeds this limit cannot be accommodated by the hardware.

★ In the signed-magnitude system:

- **Same signs:** add magnitudes and keep the common sign.
- **Different signs:** subtract the smaller magnitude from the larger and take the sign of the larger magnitude.

Example: $(+25) + (-37) = -(37 - 25) = -12$.

★ The complement of the complement restores the original number.

If the number N contains a radix point, remove it temporarily to form the r 's or $(r - 1)$'s complement, then restore it in the same relative position.

★ When subtracting $(M - N)$ where $(M < N)$ in base r using radix complements:

- If $M < N$, **no end carry is produced**.
- **Take the r 's complement of the sum and add a negative sign.**

Example: $3250 - 72532$ using 10's complement.

$M = 03250$, 10's complement of $N = 27468$. Sum = 30718.

No carry $\Rightarrow$ result = $-(10's \text{ complement of } 30718) = -69282$.

★ Subtraction of unsigned numbers can also be done using the $(r - 1)$'s complement. Because the $(r - 1)$'s complement is one less than the r 's complement, adding the minuend to the complement of the subtrahend produces a sum that is one less than the correct difference when an end carry occurs. You must perform an end-around carry by removing the end carry and adding 1 to the sum. If there is no end carry, you take the $(r - 1)$'s complement of the sum and affix a minus sign.

Example (with end carry): Subtract $X - Y$ where $X = 1010100$ and $Y = 1000011$ using 1's complement. $X = 1010100$, plus the 1's complement of Y (0111100) produces the sum 10010000. Perform an end-around carry: remove the end carry of 1 and add it to the rest of the sum ($0010000 + 1$) to get the final answer 0010001.

★ To subtract two signed binary numbers when negative numbers are in 2's-complement form: Take the 2's complement of the subtrahend (including the sign bit) and add it to the minuend (including the sign bit). A carry out of the sign-bit position is discarded.

Example (Signed 2's Complement Addition): Add +6 and -13 using 8-bit representation.

+6 $\rightarrow$ 00000110

$-13 \rightarrow$ 11110011 (This is the 2's complement of +13 which is 00001101)

```

00000110
+ 11110011
-----
11111001

```

The result 11111001 is negative (leftmost bit is 1). Its 2's complement is 00000111 (7), so the result correctly represents -7 .

★ The Hardware Advantage of the Signed-Complement System.

Binary numbers in the signed-complement system are added and subtracted by the same basic addition and subtraction rules as unsigned numbers. Therefore, computers need only one common hardware circuit to handle both types of arithmetic. This consideration has resulted in the signed-complement system being used in virtually all arithmetic units of computer systems.

Binary Codes

★ Minimum Number of Bits

An n -bit binary code can represent up to 2^n distinct combinations. To represent a set of M distinct elements, the minimum number of bits n required must satisfy the inequality:

$$2^n \geq M$$

Example: To assign a unique binary code to a standard deck of 52 playing cards, we solve for n : $2^5 = 32$ (not enough), $2^6 = 64$ (enough). Therefore, a minimum of 6 bits is required.

★ Maximum Unsigned Value

The largest unsigned integer that can be represented with n bits consists of n 1's. The decimal equivalent is calculated using the formula:

$$\text{Maximum Value} = 2^n - 1$$

Example: The largest 16-bit binary number is $2^{16} - 1 = 65,535$.

★ Binary-Coded Decimal (BCD)

To resolve the difference between the human decimal system and the computer's binary system,

we use binary codes to represent decimal digits. **BCD is a 4-bit straight binary assignment for the decimal digits 0 through 9.** A number with k decimal digits requires $4k$ bits in BCD. The binary combinations 1010 through 1111 are not used and have no meaning in BCD.

★ BCD Addition

When adding two BCD digits, they are added as if they were binary numbers. **If the binary sum is greater than or equal to 1010, or if it produces a carry, the result is an invalid BCD digit.** To correct it, **you must add 0110 (decimal 6)** to obtain the correct BCD sum and a carry for the next position (exactly 1 always).

Example (BCD Addition): Add 184 and 576 in BCD.

	0001	1000	0100	(184)
+	0101	0111	0110	(576)
Binary sum	0111	10000	1010	
Add 6		+0110	+0110	
BCD sum	0111	0110	0000	(760)

★ Other Decimal Codes

→ **Weighted Codes:** Codes where each bit position is assigned a weighting factor (e.g., the 2421 code or the 8, 4, -2, -1 code).

→ **Self-Complementing Codes:** Codes where the 9's complement of a decimal number is obtained directly by changing 1's to 0's and 0's to 1's (e.g., the 2421 and Excess-3 codes). **Excess-3 is an unweighted code** obtained by adding 3 to the corresponding binary value.

★ Gray Code

A code where **only one bit changes its value in going from one number to the next.** It is highly advantageous in converting continuous/analog data to digital form because it eliminates ambiguity and intermediate erroneous numbers during hardware transitions.

★ ASCII Character Code

The American Standard Code for Information Interchange is the standard binary code for alphanumeric characters. It uses **seven bits to code 128 characters** (including uppercase/lowercase letters, numerals, special printable characters, and control characters). ASCII characters are most often stored one per byte (8 bits).

ASCII Problem Solving Anchor Points:

- Total characters: 128. Printing characters: 94. Non-printing control characters: 34 (Codes 0-31 and 127).
- Numbers '0'-'9' begin at Hex $(30)_{16}$ to $(39)_{16}$.
- Uppercase 'A'-'Z' begin at Hex $(41)_{16}$ to $(5A)_{16}$.
- Lowercase 'a'-'z' begin at Hex $(61)_{16}$ to $(7A)_{16}$.
- **Case Toggling:** The difference between an uppercase and lowercase letter is exactly 32 (or 2^5). Therefore, to change an ASCII letter from capital to lowercase (and vice versa), you must complement the **6th bit** (bit b_6 where bits are $b_7b_6b_5b_4b_3b_2b_1$).
- **Key Punctuation:** Space is Hex $(20)_{16}$. Period is Hex $(2E)_{16}$.

★ Error-Detecting Code (Parity Bit)

To detect errors in data communication, **an eighth bit, known as a parity bit, is added to the ASCII character to make the total number of 1's either even or odd.** This method can detect one, three, or any odd combination of errors that occur during transmission.

★ Binary Cell & Registers

A **binary cell** is a device with two stable states capable of storing one bit (0 or 1). A **register** is a group of n binary cells that can store n bits of information. The content of a register can represent

binary numbers, alphanumeric characters, BCD digits, or other discrete data, depending entirely on the interpretation given to it by the system.

★ Register Transfer

A fundamental operation in digital systems consisting of **a transfer of binary information from one set of registers into another**. Data stored in memory registers can be transferred to processor registers, passed through digital logic circuits to perform an operation, and then transferred back to memory.

Binary Logic

★ Definition of Binary Logic

Equivalent to Boolean algebra, binary logic deals with binary variables (1 and 0) and three basic logical operations:

- **AND** ($\cdot$): Produces a 1 if and only if all inputs are 1.
- **OR** ($+$): Produces a 1 if any input is 1.
- **NOT** ($'$): The complement operation, which simply changes a 1 to a 0 and a 0 to a 1.

★ **Truth tables** are used to list all possible combinations of variables to show the relation between the variable values and the exact result of the operation.

x	y	$x \cdot y$ (AND)	$x + y$ (OR)	x' (NOT x)
0	0	0	0	1
0	1	0	1	1
1	0	0	1	0
1	1	1	1	0

★ Logic Gates

Logic gates are electronic circuits that operate on one or more input signals to produce an output signal. They respond to separate voltage levels that represent logic 1 and logic 0. The **AND gate** and **OR gate** can have multiple inputs, while the **NOT gate** (commonly referred to as an inverter) operates on a single input.

★ Timing Diagrams

A timing diagram visually represents logic values over time. The horizontal axis represents time segments, and the vertical axis represents voltage levels (High = 1, Low = 0).

- **How to solve timing diagram problems:** Divide the continuous wave into distinct time intervals (where inputs change state).
- For each vertical slice of time, read the binary state of each input (e.g., $a = 1, b = 0, c = 1$).
- Apply the logic gate's mathematical operation (AND, OR, NOT) to those inputs.
- Draw the resulting output wave as either High (1) or Low (0) for that duration.

تم تلخيص الوحدة الأولى

Chapter 2 : Boolean Algebra and Logic Gates

Axiomatic Definition of Boolean Algebra

Boolean algebra is an algebraic structure defined by a set of elements $B = \{0, 1\}$, together with two binary operators ($+$ and $\cdot$). It must satisfy the **Huntington postulates** (المُسَلِّمَات):

- **Closure:** Operations on $\{0, 1\}$ always result in 0 or 1.
- **Identity Elements:** 0 is the identity for $+$ ($x + 0 = x$). 1 is the identity for $\cdot$ ($x \cdot 1 = x$).
- **Commutativity:** $x + y = y + x$ and $xy = yx$.
- **Distributivity:** $x(y + z) = xy + xz$ and **Crucially:** $x + (yz) = (x + y)(x + z)$.
- **Complement:** For every x , there exists x' such that $x + x' = 1$ and $x \cdot x' = 0$.

★ Total Number of Boolean Functions:

For n binary variables, there are 2^n rows in a truth table. Since each row can output either a 0 or 1, the total number of distinct Boolean functions that can be formed is 2^{2^n} .

Worked Example: How many distinct Boolean functions can be generated using exactly 4 variables?

Solution: $2^{2^4} \rightarrow 2^{16} = 2^{16} = 65,536$ distinct functions.

Trap !!: Ordinary algebra uses subtraction and division. Boolean algebra **does not** have additive or multiplicative inverses. There is no subtraction or division!

Basic Theorems and Properties

The Duality Principle: Every algebraic expression deducible from the postulates remains valid if the operators ($+$ and $\cdot$) and identity elements (0 and 1) are interchanged.

Core Theorems for Simplification :

- **Idempotent:** $x + x = x$ and $x \cdot x = x$
- **Dominance:** $x + 1 = 1$ and $x \cdot 0 = 0$
- **Involution:** $(x')' = x$
- **Absorption Law:** $x + xy = x$ and $x(x + y) = x$
- **DeMorgan's Theorem:** $(x + y)' = x'y'$ and $(xy)' = x' + y'$
- **Consensus Theorem:** $xy + x'z + yz = xy + x'z$

Proof. Multiply yz by $(x + x')$ then we've $xy + x'z + xyz + x'yz$ By Absorption (xy with xyz yields xy) and ($x'yz$ with $x'z$ yields $x'z$) then we've $xy + x'z$. $\square$

Operator Precedence: 1. Parentheses, 2. NOT, 3. AND, 4. OR.

Advanced Simplification & Algebraic Proofs

Terminology - Literal vs. Term:

A **literal** is a single variable within a term, in either complemented or uncomplemented form. Simplification by algebraic manipulation primarily aims to reduce the number of terms, the number of literals, or both to obtain a more economical circuit.

When solving problems that ask you to "simplify to a minimum number of literals," you must write out the sequential algebraic steps using the theorems.

Worked Example 1 (De Morgan's Method): Prove algebraically that $x'y' + xy + x'y = x' + y$ by simplifying the complement of the expression.

Step 1: Let $F = x'y' + xy + x'y$. We find the complement F' :

$$F' = (x'y' + xy + x'y)'$$

$$\text{Step 2: } F' = (x'y')' \cdot (xy)' \cdot (x'y)'$$
 (De Morgan's Law: $(A + B + C)' = A'B'C'$)

$$\text{Step 3: } F' = (x + y)(x' + y')(x + y')$$
 (De Morgan's Law on individual terms)

$$\text{Step 4: } F' = [(x + y)(x + y')] \cdot (x' + y')$$
 (Rearrange to group common terms)

$$\text{Step 5: } F' = (x + yy') \cdot (x' + y')$$
 (Distributive Law: $(A + B)(A + C) = A + BC$)

$$\text{Step 6: } F' = (x + 0) \cdot (x' + y') = x(x' + y')$$
 (Null Element: $yy' = 0$)

$$\text{Step 7: } F' = xx' + xy' = 0 + xy' = xy'$$
 (Distributive Law and $xx' = 0$)

Conclusion: Since $F' = xy'$, then $F = (xy')' = x' + y$. (By De Morgan's Law)

Worked Example 2 (The Absorption Trap): Simplify $F = AB + A(B + C) + B(B + C)$

$$\text{Step 1: } F = AB + AB + AC + BB + BC$$
 (Distribute)

$$\text{Step 2: } F = AB + AC + B + BC$$
 (Idempotent: $AB + AB = AB$, and $BB = B$)

$$\text{Step 3: } F = AB + AC + B(1 + C)$$
 (Factor B from $B + BC$)

$$\text{Step 4: } F = AB + AC + B(1)$$
 (Dominance: $1 + C = 1$)

$$\text{Step 5: } F = B(A + 1) + AC$$
 (Factor B from $AB + B$)

$$\text{Step 6: } F = B + AC$$
 (Final Answer)

Worked Example 3 (Multiplying Out & Consensus): Simplify $F = (x + y)(x' + z)(y + z)$

$$\text{Step 1: } F = (xx' + xz + x'y + yz)(y + z)$$
 (Multiply first two terms)

$$\text{Step 2: } F = (0 + xz + x'y + yz)(y + z)$$
 ($xx' = 0$)

$$\text{Step 3: } F = (xz + x'y)(y + z)$$
 (**Consensus Theorem:** drops the redundant yz)

$$\text{Step 4: } F = xzy + xzz + x'yy + x'yz$$
 (Multiply out)

$$\text{Step 5: } F = xyz + xz + x'y + x'yz$$
 ($zz = z$, $yy = y$)

$$\text{Step 6: } F = xz(y + 1) + x'y(1 + z)$$
 (Factor xz and $x'y$)

$$\text{Step 7: } F = xz + x'y$$
 (Final Answer)

Finding the Complement of a Function (F'):

The Duality Method (Fastest technique for exams): First, find the *Dual* of the function (swap $+$ and $\cdot$, swap 1 and 0). Then, *complement every individual literal* (change x to x' , y' to y).

Worked Example 4: Find complement of $F = x(y'z' + yz)$.

$$\text{Step 1 (Dual): } x + (y' + z')(y + z)$$

$$\text{Step 2 (Complement literals): } F' = x' + (y + z)(y' + z')$$

Theorems with Proofs

1. De Morgan's Theorem (1): $(x + y)' = x' \cdot y'$

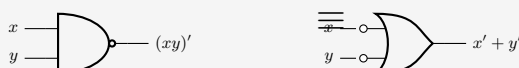
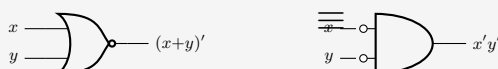
Proof. If $x = 0$ and $y = 0$ then $(x + y)' = 1$ and $x' \cdot y' = 1$, if $x = 0$ and $y = 1$ then $(x + y)' = 0$ and $x' \cdot y' = 0$, if $x = 1$ and $y = 0$ then $(x + y)' = 0$ and $x' \cdot y' = 0$, if $x = 1$ and $y = 1$ then $(x + y)' = 0$ and $x' \cdot y' = 0$. Thus in all cases we have $(x + y)' = x' \cdot y'$. $\square$

2. De Morgan's Theorem (2): $(xy)' = x' + y'$

Proof. If at least one of x or y is 0 then $xy = 0$ and thus $(xy)' = 1$ and $x' + y' = 1$, if $x = 1$ and $y = 1$ then $xy = 1$ and thus $(xy)' = 0$ and $x' + y' = 0$. Thus in all cases we have $(xy)' = x' + y'$. $\square$

★ Visual : DeMorgan's Theorems as Gate Equivalences

DeMorgan's laws are easiest to remember as the "bubble pushing" rule: *push the bubble through, and flip the gate* (AND $\leftrightarrow$ OR).

(1) NAND $\equiv$ Invert-OR(2) NOR $\equiv$ Invert-AND

Memorize: push a bubble through a gate $\Rightarrow$ flip the gate type. A bubble at the output of an AND becomes two bubbles at the inputs of an OR (and vice versa).

3. Commutative Law (1) : $x + y = y + x$

Proof. Since we're dealing with single bit (either $x = 0$ or $x = 1$), if $x = 0$ then $x + y = y$ and if $x = 1$ then $x + y = 1$ and thus in both cases we have $x + y = y + x$. $\square$

4. Commutative Law (2) : $xy = yx$

Proof. Since we're dealing with single bit, if $x = 0$ then $xy = 0$ and if $x = 1$ then $xy = y$ and thus in both cases we have $xy = yx$. $\square$

5. Associative Law (1) : $x + (y + z) = (x + y) + z$

Proof. If $x = 0$ then $x + (y + z) = y + z$ and $(x + y) + z = y + z$, if $x = 1$ then $x + (y + z) = 1$ and $(x + y) + z = 1$. Thus in all cases we have $x + (y + z) = (x + y) + z$. $\square$

6. Associative Law (2) : $x(yz) = (xy)z$

Proof. If $x = 0$ then $x(yz) = 0$ and $(xy)z = 0$, if $x = 1$ then $x(yz) = yz$ and $(xy)z = yz$. Thus in all cases we have $x(yz) = (xy)z$. $\square$

7. Distributive Law (1) : $x(y + z) = xy + xz$

Proof. By FOIL $xy + xz$ is equivalent to $x(y + z)$. $\square$

8. Distributive Law (2) : $x + (yz) = (x + y)(x + z)$

Proof. By FOIL $x \cdot x + x \cdot z + x \cdot y + y \cdot z = x + x \cdot (y + z) + y \cdot z$. Since x is less restrictive than $x(y + z)$, we simplify to $x + yz$. $\square$

9. Absorption Law (1) : $x + x \cdot y = x$

Proof. x is less restrictive than $x \cdot y$, so the result is x . $\square$

10. Absorption Law (2) : $x(x + y) = x$

Proof. x is less restrictive than $x + y$, so the result is x . □

11. **Consensus Law** : $xy + x'z + yz = xy + x'z$

Proof. Multiply yz by $(x + x')$ then we've $xy + x'z + xyz + x'yz$ By Absorption (xy with xyz yields xy) and ($x'yz$ with $x'z$ yields $x'z$) then we've $xy + x'z$. □

12. Elimination Law : $x + x' \cdot y = x + y$

Proof. $(x' \cdot (x + y')) = x'x + x'y' = 0 + x'y'$, invert and apply De Morgan to get $x + x' \cdot y = x + y$. □

13. Elimination Law : $x(x' + y) = xy$

Proof. $x \cdot x' + xy = 0 + xy = xy$. □

14. Uniting Law (1) : $xy + xy' = x$

Proof. $x(y + y') = x(1) = x$ □

15. Uniting Law (2) : $(x + y)(x + y') = x$

Proof. $x \cdot x + x(y + y') + yy' = x + x + 0 = x$ □

Canonical Forms: Minterms and Maxterms

A binary variable may appear normal (x) or complemented (x').

- **Minterm** (m_j): An AND term containing ALL variables. Evaluates to 1 for exactly one combination. (e.g., $x'y'z$ is m_1).
- **Maxterm** (M_j): An OR term containing ALL variables. Evaluates to 0 for exactly one combination. (e.g., $x + y + z'$ is M_1).
- **Relationship**: $m'_j = M_j$. (A maxterm is the exact complement of its corresponding minterm).

Visualizing Canonical Forms (3 Variables):

Inputs			Minterms (AND)		Maxterms (OR)	
x	y	z	Term	Sym.	Term	Sym.
0	0	0	$x'y'z'$	m_0	$x + y + z$	M_0
0	0	1	$x'y'z$	m_1	$x + y + z'$	M_1
0	1	0	$x'yz'$	m_2	$x + y' + z$	M_2
0	1	1	$x'yz$	m_3	$x + y' + z'$	M_3
1	0	0	$xy'z'$	m_4	$x' + y + z$	M_4
1	0	1	$xy'z$	m_5	$x' + y + z'$	M_5
1	1	0	xyz'	m_6	$x' + y' + z$	M_6
1	1	1	xyz	m_7	$x' + y' + z'$	M_7

★ **Sum of Minterms** ($\sum m$):

If a term is missing a variable, multiply it by $(v + v')$ to force it into standard form.

Worked Example (Expanding to $\sum m$): Express $F(A, B, C) = A + B'C$ as a Sum of Minterms.

$A = A(B + B') = AB + AB'$. (Still missing C)

$A = AB(C + C') + AB'(C + C') = ABC + ABC' + AB'C + AB'C'$

$B'C = B'C(A + A') = AB'C + A'B'C$

Combine and remove duplicates: $F = A'B'C + AB'C' + AB'C + ABC' + ABC = \sum(1, 4, 5, 6, 7)$.

★ Product of Maxterms ($\prod M$):

Fast Conversion Trick: If $F = \sum(1, 4, 7)$ for 3 variables (0-7), the missing terms are the maxterms:

$F = \prod(0, 2, 3, 5, 6)$.

Trap !!: Complementing Canonical Forms

To find the **complement** (F') of a canonical function, you have two choices: either swap the symbol ($\sum \leftrightarrow \prod$) and keep the **same** numbers, OR keep the same symbol and use the **missing** numbers.

Worked Example (The Complement Trick): Given $F(x, y, z) = \sum(0, 1, 2, 4, 5)$, express its complement (F').

Method 1 (Swap symbol, keep numbers): $F' = \prod(0, 1, 2, 4, 5)$.

Method 2 (Keep symbol, use missing numbers): $F' = \sum(3, 6, 7)$.

Trap !!: The Function Signature Order

The decimal number of a minterm/maxterm is strictly determined by the order of variables in the function signature! The first letter is always the Most Significant Bit (leftmost).

Example: Take the minterm $A'BC$ ($A = 0, B = 1, C = 1$).

If the signature is **F(A, B, C)**, we read it as A-B-C $\rightarrow 011 \rightarrow$ **Decimal 3**.

If the signature is **F(C, B, A)**, we read it as C-B-A $\rightarrow 110 \rightarrow$ **Decimal 6**.

Always check the parentheses before converting to binary!

Trap !! : Always ensure the count of numbers in F and F' is 2^n where n = variables count of F

Standard Forms vs Canonical Forms

Canonical Forms ($\sum m$ or $\prod M$) require EVERY term to contain EVERY variable.

Standard Forms do not require all variables in every term. They represent **two-level gate implementations** (SOP = ANDs into a final OR; POS = ORs into a final AND). Two-level implementations are preferred because they produce the least amount of signal delay.

Nonstandard Forms are factored expressions (e.g., $F = AB + C(D + E)$) that are neither SOP nor POS. They result in **three-level** (or higher) gate structures. You can convert them to Standard Form using the distributive law to reduce the gate levels.

★ Algebraic Conversion from SOP to POS:

To convert a Sum of Products (SOP) into a Product of Sums (POS) without using truth tables, rely heavily on the distributive law: $X + YZ = (X + Y)(X + Z)$.

Worked Example (SOP to POS): Convert $F = a'b + a'c' + abc$ to POS.

Step 1: Obtain sum of minterms from the expression which are $\sum(0, 2, 3, 7) \Rightarrow \prod(1, 4, 5, 6)$

Step 2: The expression now is $(a + b + c')(a' + b + c)(a' + b + c')(a' + b' + c)$:

$F = (a + b + c')(a' + b + c)(a' + b + c')(a' + b' + c)$.

Step 3: group up first and third brackets: $(a + b + c')(a' + b + c') = (b + c')$.

Step 4: group up second and last brackets: $(a' + b + c)(a' + b' + c) = (a' + c)$.

Step 5: Combine components: $F = (a' + c)(b + c')$.

Final POS Answer: **F = (a' + c)(b + c')**.

Evaluating Truth Tables from Expressions

To solve problems that ask for a truth table from a complex equation, break the equation down into sub-columns before evaluating the final function F .

Worked Example (Evaluation): Derive the truth table for $F = x(y' + z) + x'y$.

Solution strategy: Evaluate columns for x , y , z , then $(y' + z)$, then $x(y' + z)$, then $x'y$, then add them together.

x	y	z	$y' + z$	$x(y' + z)$	$x'y$	F
0	0	0	1	0	0	0
0	0	1	1	0	0	0
0	1	0	0	0	1	1
0	1	1	1	0	1	1
1	0	0	1	1	0	1
1	0	1	1	1	0	1
1	1	0	0	0	0	0
1	1	1	1	1	0	1

Drawing Logic Diagrams from Expressions

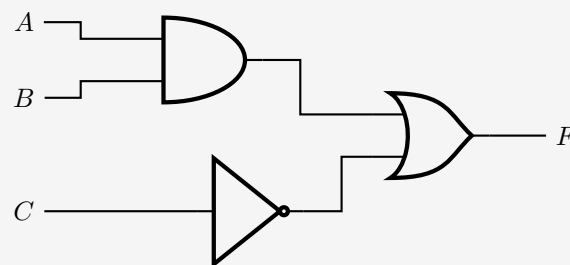
Sum of Products (SOP) results in a two-level implementation: a column of AND gates feeding into a final OR gate.

Product of Sums (POS) results in a column of OR gates feeding into a final AND gate.

Note: In standard schematic implementations, it is assumed that input variables are directly available in both their normal and complemented forms, so inverters are usually omitted from the primary diagram unless specifically required.

Definition of Implementation Level : We define the level of a circuit implementation as the number of maximum gates that any path in the circuit goes through.

Worked Example: Draw the logic diagram for $F = AB + C'$.



Other Logic Operations & Universal Gates

Out of the 16 possible binary operations for two variables, only 8 are used as standard digital logic gates. When considering the construction of logic gates, designers weigh four factors: **(1)** feasibility and economy of producing physical components, **(2)** possibility of extending the gate to more than two inputs, **(3)** basic properties like commutativity and associativity, and **(4)** the ability of the gate to implement Boolean functions alone or in conjunction with others.

★ Reference Card : The 7 Standard Logic Gates

Gate	Symbol	Expression	Output is 1 when...
AND		$x \cdot y$	both inputs are 1
OR		$x + y$	at least one input is 1
NOT		x'	input is 0 (flips the bit)
NAND		$(xy)'$	NOT both 1 (universal gate)
NOR		$(x + y)'$	both inputs are 0 (universal gate)
XOR		$x \oplus y$	inputs differ (odd # of 1's)
XNOR		$(x \oplus y)'$	inputs are equal

TT column convention: The 4-character truth-table value lists $f(00)f(01)f(10)f(11)$ in order. NOT is unary so its TT is just $f(0)f(1) = 10$. The "bubble" on NOT/NAND/NOR/XNOR is the small circle on the output — it always means **negation of whatever's behind it**.

The standard gates beyond AND ($x \cdot y$), OR ($x + y$), and NOT (x') are:

- **Buffer (Transfer):** $F = x$. Represented by a triangle symbol. It does not change the logic value but is used for power amplification (equivalent to two cascaded inverters).
- **XOR (Exclusive-OR):** Operator symbol is $\oplus$. $F = x \oplus y = xy' + x'y$. Output is 1 if inputs are *different* (odd number of 1s).
- **XNOR (Equivalence):** $F = (x \oplus y)' = xy + x'y'$. Output is 1 if inputs are *identical*.
- **NAND:** Operator symbol is $\uparrow$. $F = x \uparrow y = (xy)' = x' + y'$. (AND followed by NOT).
- **NOR:** Operator symbol is $\downarrow$. $F = x \downarrow y = (x + y)' = x'y'$. (OR followed by NOT).

Graphic Symbol Note: The small circle on the output of gates like NAND, NOR, or Inverters is formally referred to as a **bubble**, which designates the logic complement.

Note regarding other operators:

- **Inhibition:** Operator symbols are x/y (x , but not y) and y/x (y , but not x).
- **Implication:** Operator symbols are $x \supset y$ (If x , then y) and $x \subset y$ (If y , then x).

Inhibition and implication are mathematically valid operations used by logicians, but they are seldom used as standard logic gates because they are neither commutative nor associative.

Trap !!: Non-Associativity of NAND/NOR

While AND/OR are associative, NAND ($\uparrow$) and NOR ($\downarrow$) are **not**.

$$(x \downarrow y) \downarrow z \neq x \downarrow (y \downarrow z).$$

Because of this, a 3-input NOR gate is mathematically defined as $(x + y + z)'$, NOT as a cascaded sequence of 2-input NOR gates.

★ **Bitwise Operations:** To perform logic operations on multi-bit strings (e.g. $A = 1011, B = 1010$), apply the operation to each column vertically independently. (e.g. $A \text{ AND } B = 1010$).

- **Constants:** $F_0 = 0$ is officially the **Null** function, and $F_{15} = 1$ is the **Identity** function.

★ **The NAND / SOP Trick:** An expression in Sum of Products (SOP) form can be directly implemented using cascaded NAND gates. For example, $F = [(ABC)'(DE)']' = ABC + DE$ (proven via DeMorgan's Theorem).

Positive vs Negative Logic

Physical voltage levels (High/H and Low/L) are assigned to binary values (1 and 0).

- **Positive Logic:** High (H) = 1, Low (L) = 0.
- **Negative Logic:** High (H) = 0, Low (L) = 1.

Conversion Rule: The conversion from positive to negative logic is essentially changing 1s to 0s and 0s to 1s. Therefore, **changing polarity results in taking the Dual of the function**. A **polarity indicator** (represented as a small triangle on a terminal of a graphic symbol) signifies that negative logic is assumed for that signal.

Result: A positive-logic AND gate acts exactly like a negative-logic OR gate. A positive-logic NAND gate acts like a negative-logic NOR gate.

Worked Example (Polarity Conversion): Explain algebraically why a positive-logic NAND gate acts exactly like a negative-logic NOR gate.

Solution: Positive NAND is $F = (xy)' = x' + y'$. Negative logic requires taking the Dual. The dual of $x' + y'$ is $x'y'$, which is exactly the mathematical definition of a NOR gate $(x + y)'$.

Trap !!: Misleading Terminology

The terms positive and negative are misleading because both signals could physically have negative voltages (e.g., -1V and -3V). The logic type is determined solely by the *relative* amplitudes (the higher voltage is always H , the lower is always L), not by whether the voltage is mathematically greater than zero.

★ **Book Convention:** For the remainder of the text, it is assumed that all gates operate with a **positive** logic assignment unless a polarity indicator explicitly states otherwise.

Notes

★ Multiple-Input XOR (The Odd Function)

The XOR operation ($\oplus$) is commutative and associative, allowing it to be easily extended to more than two variables. A multiple-variable XOR function is famously known as an **odd function** because it evaluates to 1 if and only if an **odd number of its variables are equal to 1**.

Example: For a 3-input XOR ($F = x \oplus y \oplus z$), the output is 1 for minterms 1 (001), 2 (010), 4 (100), and 7 (111) because each of these combinations contains an odd number of 1s. Conversely, the XNOR operation extended to multiple variables is an **even function**.

★ Extension to Multiple Inputs for Logic Gates

Hardware gates are often extended to multiple inputs. AND and OR gates scale naturally due to associativity. However, because NAND and NOR are **not associative**, a multi-input NAND gate is defined strictly as a multi-input AND gate followed by a NOT gate: $(xyz)' = x' + y' + z'$. You cannot build a 3-input NAND gate by simply cascading two 2-input NAND gates.

تم تلخيص الوحدة الثانية

Chapter 3 : Gate-Level Minimization

3.1 Introduction

Gate-level minimization (التبسيط على مستوى البوابات) is the design task of finding the optimal gate-level implementation of a Boolean function. The complexity of the gates that implement a function is directly tied to the complexity of the algebraic expression.

The truth table representation is unique, but the same function can be expressed in many algebraically equivalent forms. The simpler the expression, the cheaper the circuit (fewer gates and fewer inputs per gate).

★ **Why not just use Boolean algebra?** Algebraic simplification (Chapter 2) is awkward because it lacks systematic rules — there is no clear "next step" to follow. The **Karnaugh Map (K-Map)** provides a visual, foolproof procedure that always yields a minimal expression.

★ **Goal definition:** A "simplest" expression has (1) the minimum number of terms, and (2) the minimum number of literals in each term. The simplest expression may not be *unique* — sometimes two equally simple answers exist, and either is acceptable.

3.2 The Map Method (K-Map Basics)

A **K-map** is a diagram of squares, each representing one minterm of the function. It is essentially a *pictorial truth table* where adjacent squares differ in only one variable.

The Big Idea: Two adjacent squares can be merged because:

$$m_5 + m_7 = xy'z + xyz = xz(y' + y) = xz$$

The differing variable is eliminated. Combining *more* squares eliminates *more* variables.

Two-Variable K-Map (4 squares):

		<i>y</i>		
		0	1	
<i>x</i>	0	m_0	m_1	$x'y' \quad x'y$
	1	m_2	m_3	$xy' \quad xy$

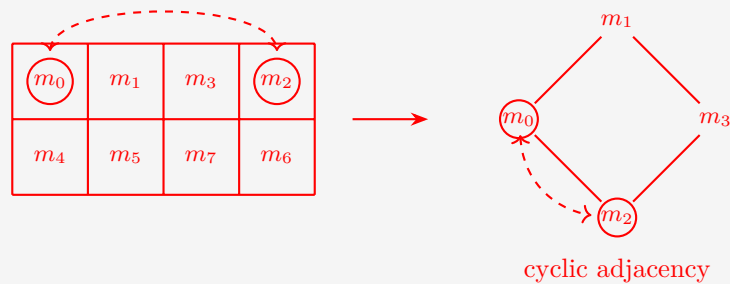
Three-Variable K-Map (8 squares): Columns are ordered in **Gray code**: 00, 01, 11, 10 (one bit flips at a time).

		$z = 1$			
		yz			
x	0	00	01	11	10
	1	00	01	11	10
$x = 1$	0	m_0	m_1	m_3	m_2
	1	m_4	m_5	m_7	m_6
		$y = 1$			

Trap !!: Gray Code Order

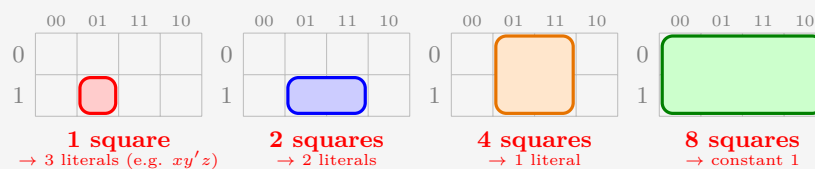
Columns are NOT in binary order (00, 01, 10, 11). They go **00, 01, 11, 10** so that horizontal neighbours always differ by exactly one bit. Same applies to rows in the 4-variable map.

Wrap-Around Adjacency: The map behaves like a cycle. The leftmost column is adjacent to the rightmost. So in the 3-variable map, m_0 & m_2 are adjacent, and so are m_4 & m_6 .

**Group Size Rule (powers of 2 only → 1, 2, 4, 8, 16, ...):**

- 1 square in 3-var map → term with 3 literals
- 2 adjacent squares → 2 literals
- 4 adjacent squares → 1 literal
- 8 adjacent squares → constant 1 (whole map)

Rule of thumb: Bigger group = simpler term. Always grow groups as large as possible.

**Trap !!: Diagonal Squares Are NOT Adjacent**

Only horizontal and vertical neighbours (and the wrap-around column/row) count as adjacent. Diagonals don't.

Reading a Term From a Group

To read a group, list only the variables that **stay constant** across the group.

- If a variable is 1 throughout the group, write it normal (x).
- If a variable is 0 throughout the group, write it complemented (x').
- If a variable changes within the group, drop it.

Worked Example 3.1 (3-var SOP): Simplify $F(x, y, z) = \Sigma(2, 3, 4, 5)$.

		yz			
		00	01	11	10
x	0	0	0	1	1
	1	1	1	0	0

$x'y$ (red box around top-right 1s)
 xy' (blue box around bottom-left 1s)

Solution: Two pairs $\rightarrow F = x'y + xy'$. Notice $x'y$: row $x=0$ stays, columns 11 and 10 both have $y=1$ but z flips $\rightarrow z$ drops.

Worked Example 3.2 (Wrap-Around): Simplify $F(x, y, z) = \Sigma(3, 4, 6, 7)$.

		yz			
		00	01	11	10
x	0	0	0	1	0
	1	1	0	1	1

xz' (blue box around bottom-left 1s)
 yz (red dashed box around top-right 1s and bottom-right 1s)

Solution: The vertical pair gives yz . The wrap-around pair m_4 & m_6 both have $x=1, z=0$, so xz' . Final: $F = yz + xz'$.

Worked Example 3.3 (Reusing a Square): Simplify $F(x, y, z) = \Sigma(0, 2, 4, 5, 6)$.

		yz			
		00	01	11	10
x	0	1	0	0	1
	1	1	1	0	1

z' (red box around top-left 1s and bottom-right 1s)
 xy' (blue box around bottom-left 1s)

Solution: The group of 4 (corners + middle) is the entire z' column $\rightarrow$ single literal z' . To cover m_5 , pair it with m_4 to give xy' . Squares can be reused in multiple groups. $F = z' + xy'$.

Worked Example 3.4 (Plotting From SOP):

For $F = A'C + A'B + AB'C + BC$, (a) express as sum of minterms and (b) find minimal SOP.

Plotting: $A'C$ covers row $A = 0$, columns $z = 1 \rightarrow$ squares m_1, m_3 . $A'B$ covers row $A = 0$, $B = 1$ part $\rightarrow m_2, m_3$. $AB'C$ is single minterm m_5 . BC covers $B = 1, C = 1 \rightarrow m_3, m_7$. Total minterms: $\{1, 2, 3, 5, 7\}$.

		BC			
		00	01	11	10
A	0	0	1	1	1
	1	0	1	1	0

Result: $F = \Sigma(1, 2, 3, 5, 7) = C + A'B$.

3.3 Four-Variable K-Map

The 4-variable map has 16 squares. Both rows and columns use Gray code (00, 01, 11, 10). Wrap-around now occurs in **both** directions: top↔bottom and left↔right.

		yz			
		00	01	11	10
wx	00	m_0	m_1	m_3	m_2
	01	m_4	m_5	m_7	m_6
	11	m_{12}	m_{13}	m_{15}	m_{14}
	10	m_8	m_9	m_{11}	m_{10}

Group Sizes for 4-Variable Map:

- 1 square $\rightarrow$ 4 literals
- 2 squares $\rightarrow$ 3 literals
- 4 squares $\rightarrow$ 2 literals
- 8 squares $\rightarrow$ 1 literal
- 16 squares $\rightarrow$ constant 1

Worked Example 3.5: Simplify $F(w, x, y, z) = \Sigma(0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14)$.

		yz			
		00	01	11	10
wx	00	1	1	0	1
	01	1	1	0	1
	11	1	1	0	1
	10	1	1	0	0

Reading the groups:

- Group of 8 (red, two leftmost columns) $\rightarrow y'$. Column 00 has $y=0, z=0$; column 01 has $y=0, z=1$. Only y stays $\rightarrow y'$.
- Group of 4 (purple, the right column joined with the leftmost column via wrap-around in middle two rows) $\rightarrow xz'$.

- Group of 4 (blue, the four corner-adjacent squares of the top-right region, joining with wrap-around) $\rightarrow w'z'$.

Result: $F = y' + w'z' + xz'$.

Worked Example 3.6 (The 4-Corners Trick): Simplify $F = A'B'C' + B'CD' + A'BCD' + AB'C'$.

Plotting the minterms: $\{0, 1, 2, 6, 8, 9, 10\}$. The four corners (m_0, m_2, m_8, m_{10}) form a single group of 4 because of dual wrap-around $\rightarrow B'D'$.

		CD			
		00	01	11	10
AB	00	1	1	0	1
	01	0	0	0	1
	11	0	0	0	0
	10	1	1	0	1

$B'D'$ (4 corners $\rightarrow$ 1 group)

Reading: Four corners $\rightarrow B'D'$. The top and bottom row left-pair $\rightarrow B'C'$. Single m_6 pairs with m_2 (already used) for $A'CD'$. $F = B'D' + B'C' + A'CD'$.

Prime Implicants & Essential Prime Implicants

Implicant: Any product term that, when equal to 1, makes $F = 1$ (any rectangle of 1's) i.e. any minterm (since sum of 1 and any combination of 0's and 1's ins 1).

Prime Implicant (PI): An implicant that cannot be merged with another adjacent group to form a larger one. (A prime implicant is a "maximal" rectangle.)

Essential Prime Implicant (EPI): A PI that is the *only* PI covering some particular minterm.

★ Standard Procedure for Minimal SOP:

1. Plot all 1's on the K-map.
2. Identify all the prime implicants (largest possible rectangles covering each 1).
3. Mark the **essential** prime implicants — every minterm covered by only one PI.
4. Choose additional PIs (smallest set possible) to cover the remaining minterms.

Trap !!: Don't redundant-cover

A group whose minterms are all already covered by other (already-chosen) groups is redundant. Drop it.

Example: For $F(A, B, C, D) = \Sigma(0, 2, 3, 5, 7, 8, 9, 10, 11, 13, 15)$, the two essential PIs are BD (covers m_5, m_7, m_{13}, m_{15}) and $B'D'$ (covers m_0, m_2, m_8, m_{10}). Remaining minterms $\{3, 9, 11\}$ can be covered by any 2 of: $\{CD, B'C, AD, AB'\}$, giving multiple equally-minimal answers.

Trap !!: Five-Variable Maps

A 5-var map has 32 squares (two stacked 4-var maps) and 6-var has 64. The geometry gets messy. The textbook does **not** use them — for ≥ 5 variables, use computer tools (Quine-McCluskey, Espresso).

3.4 Product-of-Sums Simplification

To get a minimal POS form: **group the 0's**, read those groups (which gives F' in SOP), then complement using DeMorgan's theorem.

★ **Reading 0-groups:** The opposite convention applies — variables that stay constant become summed; a variable that is 0 throughout enters as x , a variable that is 1 throughout enters as x' . (Equivalently: read it as SOP for F' , then DeMorgan.)

Worked Example 3.7: Simplify $F(A, B, C, D) = \Sigma(0, 1, 2, 5, 8, 9, 10)$ in (a) SOP and (b) POS.

(a) Group the 1's $\rightarrow F = B'D' + B'C' + A'C'D$.

(b) Group the 0's $\rightarrow F' = AB + CD + BD'$.

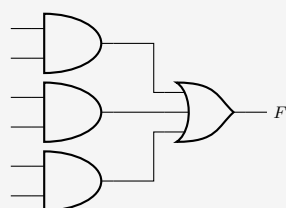
Apply DeMorgan: $F = (F')' = (AB + CD + BD')'$.

Easier route: complement F' via DeMorgan directly:

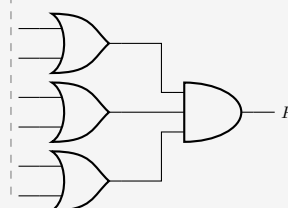
$$F = (AB + CD + BD')' = (AB)'(CD)'(BD')' = (A' + B')(C' + D')(B' + D).$$

★ **Two Equivalent Two-Level Implementations:**

SOP (ANDs $\rightarrow$ OR)



POS (ORs $\rightarrow$ AND)



Both forms are **two-level implementations** (input $\rightarrow$ first gate type $\rightarrow$ final gate $\rightarrow$ output). Two-level circuits are preferred because they minimize gate-propagation delay.

★ **Plotting POS in K-Map:** Given $F = (A + B + C)(B' + D)$, first take complement $F' = A'B'C' + BD'$, plot F' 's minterms as 0's, fill remaining with 1's. Then proceed as usual.

3.5 Don't-Care Conditions

Incompletely specified function: For some input combinations, the output is irrelevant — either because those inputs never occur (e.g., the 6 unused codes in BCD) or because the output isn't observed in those cases. We mark such cells with X (or d) for "don't care".

★ **Strategy:** An X may be treated as either 0 or 1, whichever gives a larger group (= simpler term). **Never** create a group just to cover an X — only include X 's when they enlarge an existing group of 1's.

Notation: $F(w, x, y, z) = \Sigma(1, 3, 7, 11, 15)$ with $d(w, x, y, z) = \Sigma(0, 2, 5)$.

Worked Example 3.8: Simplify F above.

		yz			
		00	01	11	10
wx	00	X	1	1	X
	01	0	X	1	0
	11	0	0	1	0
	10	0	0	1	0

yz (group of 4)

$w'x$ (uses X's at m_0, m_2)

Two equally valid solutions:

- Including X at $m_0, m_2 \rightarrow F = yz + w'x'$ (top row, treating those X as 1)
- Including X at $m_5 \rightarrow F = yz + w'z$ (column 01 in top half)

Either expression is correct because they only differ in unused input combinations.

Trap !!: Don't Force-Cover X's

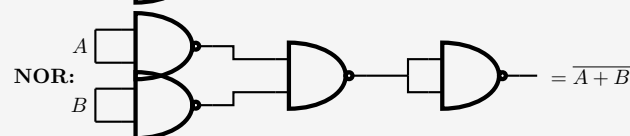
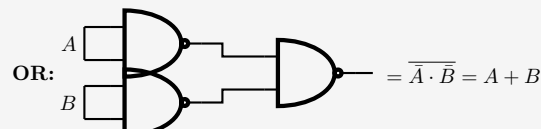
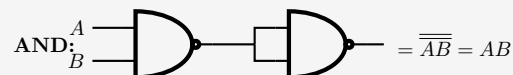
If grouping a particular X requires creating an extra term, leave it as 0 — it costs gates without simplifying anything.

3.6 NAND & NOR Implementation

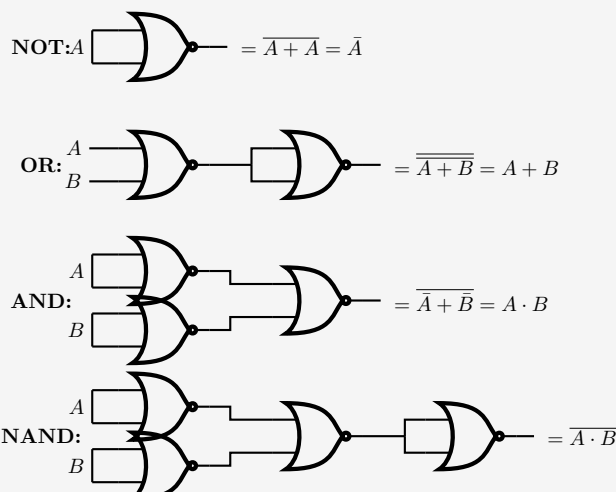
NAND and NOR gates are easier to fabricate in IC technology and are the building blocks of practical digital logic. Both are **universal gates**: any Boolean function can be built using only NAND, or only NOR.

Why NAND is universal — derivation of NOT, AND, OR:

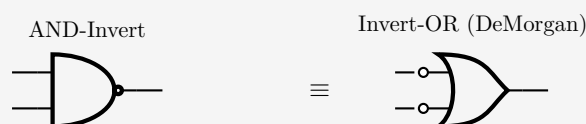
- **NOT:** $(x \cdot x)' = x'$ (one-input NAND = inverter)
- **AND:** $((xy)')' = xy$ (NAND followed by NAND-inverter)
- **OR:** $(x' \cdot y')' = x + y$ (DeMorgan: invert inputs, then NAND)



Why NOR is universal: dual derivation — NOT = $(x + x)'$, OR = $((x + y)')'$, AND = $(x' + y')'$.



Two Equivalent NAND Symbols (used to read circuits cleanly):



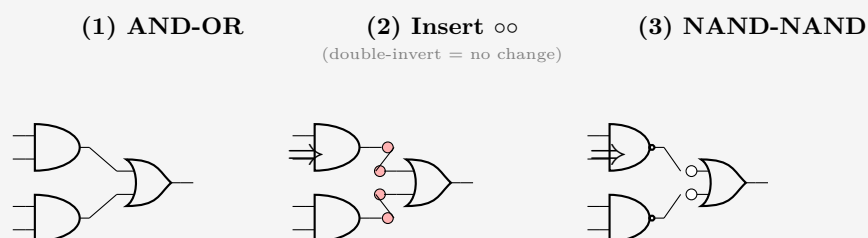
Both symbols represent the same NAND gate. The Invert-OR form follows from $(xy)' = x' + y'$ — it makes "bubble matching" easier when reading multi-level NAND circuits.

Two-Level NAND-NAND (implements SOP):

Take an AND-OR (SOP) circuit. Replace every AND with NAND (AND-invert symbol) and replace the OR with NAND (Invert-OR symbol). Bubbles on the wires cancel pairwise, leaving the same function.

For $F = AB + CD$: $F = ((AB)') \cdot (CD)')' = AB + CD$. The whole circuit becomes *four NAND gates*.

★ Visual Transformation (AND-OR → NAND-NAND):



Reading: Step (1) shows the SOP circuit. In step (2) we insert pairs of bubbles on each wire — two bubbles in series cancel ($x'' = x$), so the function is unchanged. In step (3) we regroup: each bubble on the AND output combines with the AND to form a NAND, and the bubbles on the OR inputs make it the Invert-OR form (DeMorgan-equivalent NAND). Result: four NAND gates implementing the same SOP.

Procedure for 2-level NAND from SOP:

1. Simplify F to SOP form (using K-map).
2. For each product term with ≥ 2 literals, draw a NAND with those literals as inputs (first level).
3. Draw one final NAND in the second level whose inputs are the outputs of the first-level NANDs.

4. For a single-literal term: feed it directly as $literal'$ to the second NAND, or insert a NAND-inverter on the literal.

Worked Example 3.9: Implement $F(x, y, z) = \Sigma(1, 2, 3, 4, 5, 7)$ with NAND only.

Step 1 (K-map): $F = xy' + x'y + z$.

Step 2: Build NAND for xy' , NAND for $x'y$, and an inverter on z (since it's a single literal). Final NAND combines all three.

NOR Implementation (Dual):

- NOR-NOR implements POS. Take an OR-AND circuit, replace each OR with NOR (OR-Invert) and the AND with NOR (Invert-AND).
- Single-literal terms going into the second-level NOR must be **complemented**.

Multi-Level NAND/NOR — General Conversion:

1. Convert each AND $\rightarrow$ NAND with AND-Invert symbol.
2. Convert each OR $\rightarrow$ NAND with Invert-OR symbol.
3. For every *uncompensated* bubble (no matching bubble on the same wire), insert an inverter or complement the input literal.

Trap !!: NAND-only doesn't always need a clean SOP first

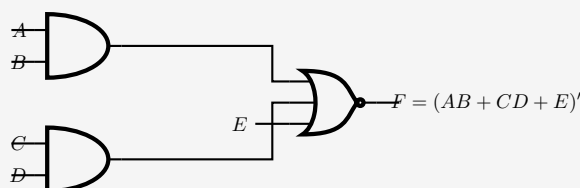
For multi-level designs, sometimes a factored (non-SOP) form gives fewer gates. Don't blindly expand to two-level — count gates after each form.

3.7 Other Two-Level Implementations

With four gate types (AND, OR, NAND, NOR) available for each level, there are $4 \times 4 = 16$ possible two-level forms. **8 are degenerate** (collapse to a single operation, e.g. AND-AND = AND of all inputs). The **8 nondegenerate forms** are:

Form	Function
AND-OR	SOP
NAND-NAND	SOP
OR-AND	POS
NOR-NOR	POS
AND-NOR	AND-OR-INVERT (AOI)
NAND-AND	AND-OR-INVERT (AOI)
OR-NAND	OR-AND-INVERT (OAI)
NOR-OR	OR-AND-INVERT (OAI)

AND-OR-INVERT (AOI): Implements $F = (AB + CD + E)'$. Used when the *complement* of F has a simpler SOP (group the 0's of the original F).



OR-AND-INVERT (OAI): Implements $F = [(A + B)(C + D)E]'$. Used when F' has a simpler POS.

Wired Logic: On certain IC families (open-collector TTL NAND, ECL NOR), tying outputs together with a wire physically performs an extra logical operation *free of charge*:

- Wired-AND (open-collector NAND) $\rightarrow$ AND-OR-INVERT
- Wired-OR (ECL NOR outputs) $\rightarrow$ OR-AND-INVERT

★ **When to use AOI/OAI:** When grouping the 0's of F on the K-map gives a simpler expression than grouping the 1's, AOI/OAI is the cheaper implementation.

Worked Example 3.10: Suppose for some $F(x, y, z)$, grouping the 0's gives $F' = xy + x'y'z'$ (a 2-term SOP with only 5 literals total), while grouping the 1's gives $F = xy' + x'z + yz$ (3 terms, 6 literals).

- **AOI (AND-NOR):** feed xy and $x'y'z'$ (formed by AND gates at level 1) into a NOR. Output: $F = (xy + x'y'z')'$. Total: 4 gates.
- **Standard SOP:** 3 ANDs + 1 OR = 4 gates anyway, but uses 6 literals worth of input wiring vs. 5.
- **Decision:** AOI wins on input count and is preferred when ICs are AOI-cell-based.

Rule: If groups-of-0's are simpler than groups-of-1's, choose AOI. If groups-of-0's of F' (i.e., POS-form F') are simplest, choose OAI.

3.8 Exclusive-OR Function

The **XOR** ($\oplus$) operator is defined by:

$$x \oplus y = xy' + x'y$$

Output is 1 when inputs *differ*. Its complement, the **XNOR** (or "equivalence" $\odot$), is:

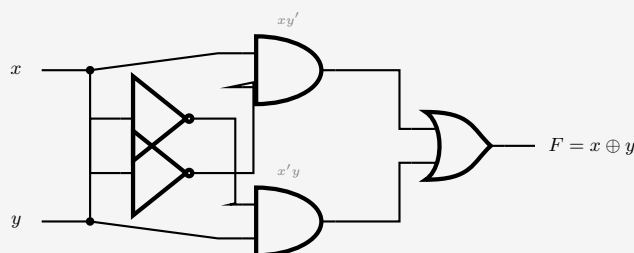
$$(x \oplus y)' = xy + x'y'$$

XNOR is 1 when inputs are *equal*.

★ **Essential XOR Identities (memorize these):**

$x \oplus 0 = x$	$x \oplus 1 = x'$
$x \oplus x = 0$	$x \oplus x' = 1$
$x \oplus y = y \oplus x$ (<i>commutative</i>)	$(x \oplus y) \oplus z = x \oplus (y \oplus z)$ (<i>associative</i>)
$x \oplus y' = (x \oplus y)' = x' \oplus y$	<i>Complementing one input flips XOR.</i>

Implementation with conventional gates (5 gates: 2 NOT, 2 AND, 1 OR):



Implementation with NAND only (4 NANDs): A clever trick — see textbook Fig. 3.30(b). The first NAND computes $(xy)'$, then 3 more NANDs combine it to give $xy' + x'y$.

Multi-Input XOR — The Odd Function:

$$A \oplus B \oplus C = \Sigma(1, 2, 4, 7)$$

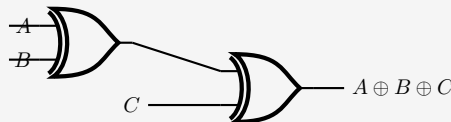
This is 1 iff an **odd number** of inputs are 1. The general n -input XOR (**odd function**) outputs 1 for those minterms whose binary value has an odd number of 1's.

Plot of 3-variable odd function on a K-map: the 1's form a checkerboard pattern (each 1 has no adjacent 1's) — this is why XOR functions *cannot be simplified* on a K-map.

		BC			
		00	01	11	10
A	0	0	1	0	1
	1	1	0	1	0

Checkerboard
→ no adjacency
→ no simplification

The **even function** (XNOR of multiple inputs) is the complement: 1 iff an **even number** of inputs are 1 (including zero 1's).

Building a 3-input Odd Function with 2-input XOR gates:**Trap !!: Multi-Input XOR Hardware**

A 3-input XOR *gate* is hard to fabricate directly. In practice, a tree of 2-input XORs is used (associativity makes any tree shape valid).

Parity Generation & Checking

A **parity bit** is appended to a binary message so that the total number of 1's (data + parity) has a fixed parity (even or odd). It detects single-bit errors during transmission.

Even-Parity Generator (3-bit message x, y, z + parity P):

$$P = x \oplus y \oplus z$$

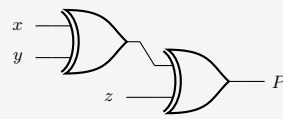
This makes the total count of 1's in (x, y, z, P) even. (For odd parity, take the complement: $P = (x \oplus y \oplus z)'$.)

Even-Parity Checker: XOR all received bits including the parity bit:

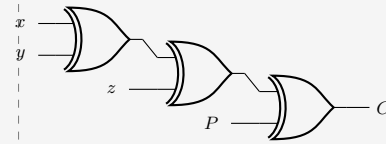
$$C = x \oplus y \oplus z \oplus P$$

- $C = 0 \rightarrow$ no error (even number of 1's preserved).
- $C = 1 \rightarrow$ error detected (odd number of 1's $\rightarrow$ one bit flipped).

(a) Even-Parity Generator



(b) Even-Parity Checker



★ **Reusable Hardware Trick:** The *same* 4-input XOR circuit can be used as both generator (feed $P = 0$) and checker (feed received P). This works because $A \oplus B \oplus C \oplus 0 = A \oplus B \oplus C$.

Trap !!: Single-Error Detection Only

Parity catches an **odd** number of bit flips. If two bits flip simultaneously, parity is preserved and the error goes undetected. Real systems use Hamming or CRC codes for stronger protection.

Summary

- **K-Map columns/rows are Gray-coded:** 00, 01, 11, 10. **Wrap-around** always applies.
- **Group sizes must be powers of 2.** Bigger group = simpler term.
- **SOP** → group 1's. **POS** → group 0's then DeMorgan.
- **Don't-cares (X) are joker squares.** Use only when they enlarge an existing group.
- **Essential PI:** the *only* prime implicant covering some minterm. Always include in answer.
- **NAND-NAND** $\equiv$ **AND-OR (SOP).** **NOR-NOR** $\equiv$ **OR-AND (POS).**
- **AOI = AND-NOR or NAND-AND.** Realizes $(SOP)'$ — useful when 0's group simpler than 1's.
- **OAI = OR-NAND or NOR-OR.** Realizes $(POS)'$.
- **XOR identities:** $x \oplus 0 = x$, $x \oplus 1 = x'$, $x \oplus x = 0$, $x \oplus x' = 1$.
- **Multi-input XOR = Odd function.** 1 iff odd number of 1's. Cannot be K-map simplified (checkerboard).
- **Even Parity:** $P = x \oplus y \oplus z$. Same XOR tree works for both generator and checker.

تم تلخيص الوحدة الثالثة

Chapter 4 : Combinational Logic

4.1 Introduction

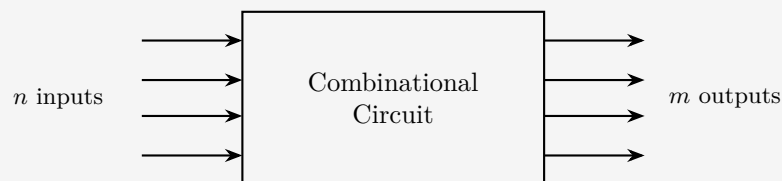
Digital circuits split into two families:

- **Combinational** (دارات تجميعية) : outputs depend *only* on the present input combination. No memory. Fully specified by a set of Boolean functions / a truth table.
- **Sequential** (دارات متتابعة) : outputs depend on present inputs *and* on stored state (memory elements). Behaviour is a time-sequence of inputs and internal states. (Covered in Chapters 5 & 8.)

★ **Scope of this chapter:** (1) **Analyze** a given combinational circuit, (2) **design** a circuit to meet a spec, and (3) survey the standard building blocks — adders, subtractors, comparators, decoders, encoders, multiplexers. These exist as MSI chips and as standard cells inside VLSI / ASIC designs.

4.2 Combinational Circuits

A combinational circuit is an **interconnection of logic gates** with **no feedback paths** and **no memory elements**. It transforms n input bits into m output bits.



For n inputs there are 2^n possible combinations. The circuit can be described by:

- a **truth table** of 2^n rows $\times$ m output columns, **or**
- m **Boolean functions**, one per output, each in terms of the n inputs.

Each input/output is physically an analog voltage interpreted as logic 0 or 1. If storage registers are wrapped around the gates, the whole assembly becomes sequential, not combinational.

4.3 Analysis Procedure

Goal: Given a logic diagram, recover what it does — as Boolean expressions, a truth table, or a recognised standard function.

Step 0 (mandatory): Confirm the circuit is combinational. **Any feedback loop** $\Rightarrow$ **sequential** — stop and use sequential-analysis methods instead.

Procedure (Boolean-expression route):

1. Label every gate output that depends *only on inputs* with symbols $T_1, T_2, \dots$ and write its Boolean expression.
2. Label gates that depend on already-labeled symbols; write their expressions.
3. Repeat until you reach the circuit outputs.

4. Back-substitute so every output is expressed purely in input variables.

Procedure (truth-table route): List all 2^n input rows, then fill columns gate-by-gate using previously-filled columns until output columns are complete.

★ **Textbook example (Fig. 4.2):** A circuit with inputs A, B, C and outputs F_1, F_2 analyses to

$$F_2 = AB + AC + BC, \quad F_1 = A'B'C + A'BC' + AB'C' + ABC$$

Inspection of the truth table reveals this is exactly a **full adder**: $F_1 = \text{sum}$, $F_2 = \text{carry}$. *Recognising standard functions from their truth tables is a skill worth practicing.*

Trap !!: Logic simulation is not analysis

Simulation only checks behaviour for the patterns you feed it — useful for *verification*, but it cannot derive a Boolean expression from a circuit. For analysis, follow the procedure above.

4.4 Design Procedure

The mirror image of analysis: go from specification $\rightarrow$ circuit.

1. Determine input/output count from the spec; assign symbols.
2. Build the truth table.
3. Simplify each output (algebra, K-map, or CAD tool).
4. Draw the diagram and verify (by hand or simulation).

Real-world design constraints you must balance:

- Total number of gates.
- **Fan-in** (inputs per gate) — limited by technology.
- **Fan-out** (driving capability) — how many gate inputs a single output can drive.
- **Propagation delay** through the longest path.
- Wiring / interconnect count.

No single “simplest” form is universally best — the dominant constraint dictates the choice.

Code Conversion Example: BCD $\rightarrow$ Excess-3

Codes change between systems; a **code converter** bridges them. Excess-3 of a decimal digit = BCD + 3.

Inputs A, B, C, D (BCD); **Outputs** w, x, y, z (Excess-3). Truth table covers digits 0–9; combinations 1010–1111 are **don't-cares** (illegal BCD).

A	B	C	D	w	x	y	z	dec
0	0	0	0	0	0	1	1	0
0	0	0	1	0	1	0	0	1
0	0	1	0	0	1	0	1	2
0	0	1	1	0	1	1	0	3
0	1	0	0	0	1	1	1	4
0	1	0	1	1	0	0	0	5
0	1	1	0	1	0	0	1	6
0	1	1	1	1	0	1	0	7
1	0	0	0	1	0	1	1	8
1	0	0	1	1	1	0	0	9

K-map simplification (with don't-cares to enlarge groups):

$$z = D'$$

$$y = CD + C'D' = CD + (C + D)'$$

$$x = B'C + B'D + BC'D' = B'(C + D) + B(C + D)'$$

$$w = A + BC + BD = A + B(C + D)$$

★ **Shared sub-expression trick:** The term $(C + D)$ appears in w , x , y . Compute it once and reuse it. A flat two-level SOP needs 7 AND + 3 OR gates; the shared-OR three-level version needs only 4 AND + 4 OR + 1 inverter. **Multi-output systems often share gates** — a key economy of multi-level design.

4.5 Binary Adder–Subtractor

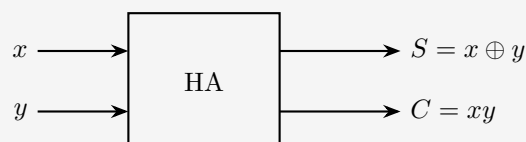
Built hierarchically: **half adder** → **full adder** → **n -bit ripple-carry adder** → **adder/subtractor**.

Half Adder (HA)

Adds **two** bits. Outputs sum S , carry C .

x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$$S = x \oplus y, \quad C = xy$$



Full Adder (FA)

Adds **three** bits: x , y , and the carry-in z from the previous position.

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

From K-maps:

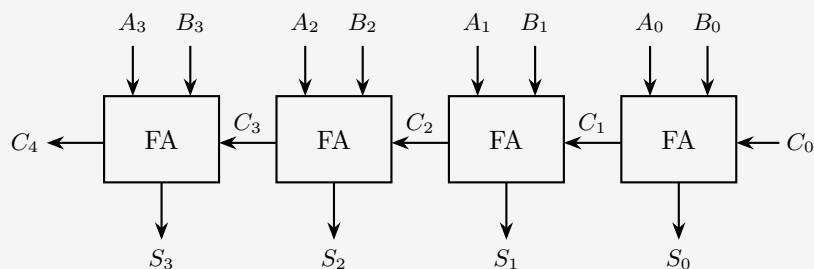
$$S = x \oplus y \oplus z \quad (\text{odd / parity function})$$

$$C = xy + xz + yz \quad (\text{majority function})$$

★ **Two half-adders + OR = one full adder.** Chain a first HA on inputs (x, y) to a second HA on $(x \oplus y, z)$. The second HA's top output is S . The two HAs each emit a carry (xy and $(x \oplus y)z$); OR them together to get C .

Ripple-Carry Adder (n -bit)

Cascade n full adders: each FA's carry-out feeds the next FA's carry-in. The least-significant carry-in $C_0 = 0$ for pure addition.



Worked Example. $A = 1011$, $B = 0011$:

pos i	3	2	1	0	
carry C_i	0	1	1	0	
A_i	1	0	1	1	
B_i	0	0	1	1	
S_i	1	1	1	0	$(C_4 = 0)$

★ **Why iterative design wins:** A classical 9-input ($4+4+1$ carry) 4-bit adder truth table has $2^9 = 512$ rows. Cascading 4 identical FA blocks bypasses that explosion entirely.

Carry Propagation & Carry Lookahead (CLA)

The problem: In ripple-carry, S_3 cannot stabilise until C_3 arrives — but C_3 waits on C_2 , which waits on C_1 ...Each stage adds **2** gate-levels of delay; an n -bit ripple adder takes **$2n$** gate-levels for the carry to settle.

CLA idea: Define, per bit position:

$$P_i = A_i \oplus B_i \quad (\text{carry propagate})$$

$$G_i = A_i B_i \quad (\text{carry generate})$$

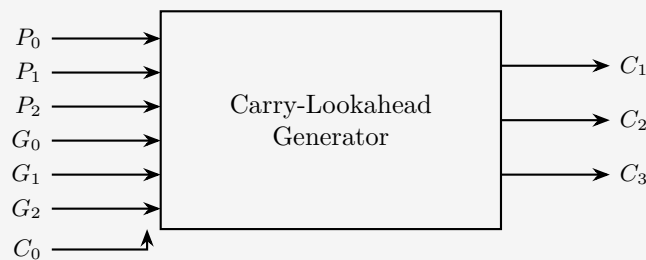
Then $S_i = P_i \oplus C_i$ and $C_{i+1} = G_i + P_i C_i$. Unrolling the recursion:

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$$

$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

Each C_i is now a single sum-of-products $\Rightarrow$ implementable in **2 gate-levels** (AND-then-OR). **All carries appear simultaneously** after a constant delay, independent of n .



Trade-off: CLA is much faster but uses more (and wider) gates. Real CPUs use hybrid schemes (block-CLA, carry-select) for very wide adders.

Binary Subtractor and Combined Adder/Subtractor

Key identity: $A - B = A + (2\text{'s complement of } B) = A + B' + 1$.

So a subtractor is just an adder with:

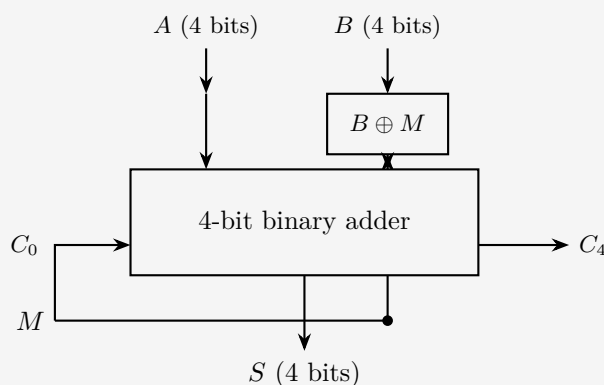
- every B_i replaced by B'_i (NOT it), and
- carry-in set to 1.

★ **One circuit, two operations:** Put an **XOR** on each B_i line with a mode bit M :

$$B_i \oplus M = \begin{cases} B_i & \text{if } M = 0 \text{ (add)} \\ B'_i & \text{if } M = 1 \text{ (subtract)} \end{cases}$$

Tie $C_0 = M$. Then the same hardware computes:

- $M = 0$: $A + B$.
- $M = 1$: $A + B' + 1 = A - B$ (in 2's complement).



The “ $B \oplus M$ ” block is just four XOR gates (one per bit) sharing the mode line M .

Overflow Detection

Overflow (فيض) = the true result needs more bits than the register has.

- **Unsigned numbers:** overflow $\Leftrightarrow$ final carry-out $C_n = 1$.
- **Signed (2's complement) numbers:** the carry-out is **NOT** a reliable overflow indicator (the sign bit is part of the number). Instead:

$$V = C_{n-1} \oplus C_n$$

i.e. overflow happens iff the carry *into* the sign bit differs from the carry *out* of it.

Why it works. Overflow in signed addition only occurs when adding two same-sign numbers gives the opposite sign. That sign flip happens exactly when those two carries disagree.

★ **Rule of thumb:** adding a positive and a negative number *cannot* overflow — the magnitude shrinks. Only same-sign additions can overflow.

Trap !!: Don't confuse C and V

The same circuit produces both. Interpret C for unsigned operations, V for signed. Hardware doesn't know which one you meant — it's the programmer's interpretation that matters.

4.6 BCD (Decimal) Adder

A BCD adder takes two BCD digits + carry-in (so $4 + 4 + 1 = 9$ inputs) and produces a BCD digit + carry-out.

Range issue: two BCD digits + carry $\leq 9 + 9 + 1 = 19$. A pure 4-bit binary adder produces values 0 through 19 (in 5 bits, $KZ_8Z_4Z_2Z_1$), but only 0–9 are valid BCD. The remaining 10–15 are invalid BCD codes; 16–19 already have $K = 1$.

Correction rule: If binary sum > 9 , add binary 0110 (=6) to land on the correct BCD digit and assert the BCD carry-out.

$$C_{\text{out}} = K + Z_8Z_4 + Z_8Z_2$$

The Z_8Z_4 and Z_8Z_2 terms catch values 10–15 where the high bit is set but the carry isn't.



$$x_i = A_i B_i + A_i' B_i' \quad (\text{XNOR: } x_i = 1 \text{ iff } A_i = B_i)$$

Equality output: all bits must match.

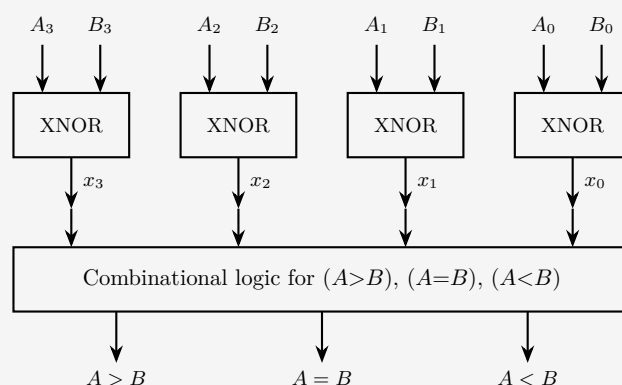
$$(A = B) = x_3x_2x_1x_0$$

Inequality outputs: scan from MSB; the first mismatch decides.

$$(A > B) = A_3B'_3 + x_3A_2B'_2 + x_3x_2A_1B'_1 + x_3x_2x_1A_0B'_0$$

$$(A < B) = A'_3B_3 + x_3A'_2B_2 + x_3x_2A'_1B_1 + x_3x_2x_1A'_0B_0$$

Top-level structure of a 4-bit comparator. Four XNOR cells produce the per-bit equality bits x_0, x_1, x_2, x_3 ; these feed a combinational block that also receives A_i and B_i to compute the three outputs.



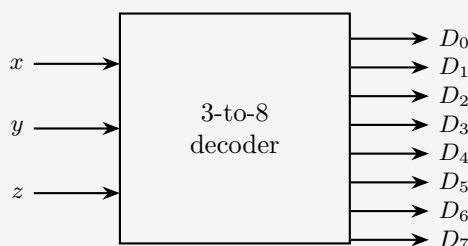
(The combinational block also needs A_i and B_i themselves to form the $A_iB'_i$ and A'_iB_i terms; those wires are omitted here for clarity.)

★ **Why this scales:** the structure is regular and recursive — extending to n bits just adds more terms of the same shape. No truth-table blow-up.

4.9 Decoders

A **decoder** (مفكّك ترميز) converts n coded input lines into up to 2^n unique output lines. Each input pattern asserts exactly one output. The asserted output corresponds to one **minterm** of the inputs.

Block view of a $3 \rightarrow 8$ decoder. Inputs x, y, z ; outputs $D_0 \dots D_7$ where $D_i = m_i$.



Truth table: row i has only $D_i = 1$.

Internal structure. Each output is an AND gate that picks one specific combination of complemented/uncomplemented input lines. A 3-to-8 decoder uses 3 inverters (supplying x', y', z') and 8 three-input AND gates — one per minterm.

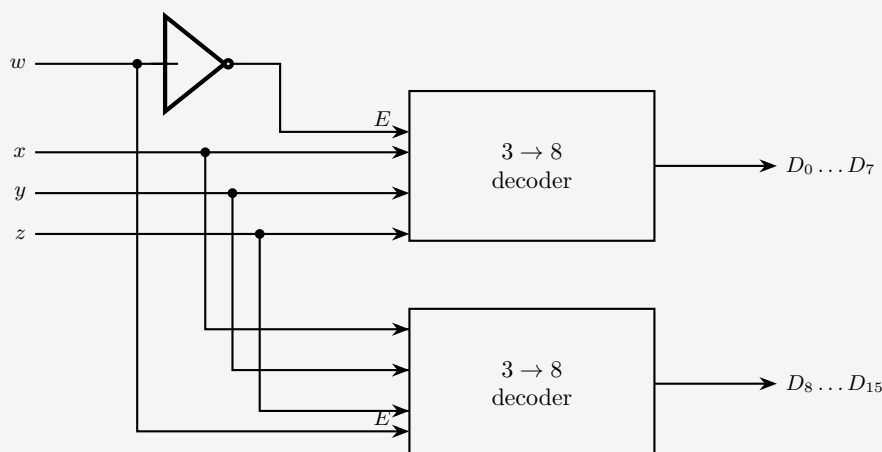
Example: $D_0 = x'y'z'$, $D_1 = x'y'z$, $D_2 = x'yz'$, ..., $D_7 = xyz$.

Enable input. A decoder usually has an enable line E :

- E active $\Rightarrow$ normal decoding.
- E inactive $\Rightarrow$ all outputs forced to inactive (the chip is “off”).

★ **Decoder = Demultiplexer.** Feed the data on the enable line and use the address inputs as selection: a n -to- 2^n decoder doubles as a 1-to- 2^n demultiplexer.

Expansion (cascading decoders). A $4 \rightarrow 16$ decoder is built from two $3 \rightarrow 8$ decoders. The high-order bit w goes (inverted) to the top decoder's enable and (directly) to the bottom decoder's enable. When $w = 0$, top is active; when $w = 1$, bottom is.



Implementing Boolean Functions with a Decoder

A decoder generates all 2^n minterms. Any Boolean function in sum-of-minterms form is just an OR of selected decoder outputs.

★ **Recipe:** For each output of a multi-output combinational circuit, OR together the decoder lines listed in its minterm expansion.

Example: full adder using a $3 \rightarrow 8$ decoder.

$$S(x, y, z) = \Sigma(1, 2, 4, 7)$$

$$C(x, y, z) = \Sigma(3, 5, 6, 7)$$

Connect x, y, z to the decoder inputs; OR lines 1, 2, 4, 7 to get S ; OR lines 3, 5, 6, 7 to get C .

Cost optimisation. A function with k minterms needs a k -input OR. If $k > 2^n/2$, its *complement* has fewer minterms — implement F' and add an inverter (or use a NOR), saving gate inputs.

4.10 Encoders

An **encoder** (مرمّز) is the inverse of a decoder: 2^n inputs $\rightarrow n$ output lines carrying the binary code of whichever input is asserted.

Octal-to-binary encoder (8 inputs $D_0 \dots D_7$, outputs xyz):

$$z = D_1 + D_3 + D_5 + D_7$$

$$y = D_2 + D_3 + D_6 + D_7$$

$$x = D_4 + D_5 + D_6 + D_7$$

Implementable with three OR gates only.

Trap !!: Two ambiguities of a plain encoder

1. **Multiple active inputs** produce a garbage output (e.g. D_3 and D_6 active $\Rightarrow$ output 111 = 7, which is neither).
2. **All inputs inactive** looks the same as D_0 active (both give output 000).

Both are fixed by adding a **priority scheme** and a **valid bit**.

Priority Encoder

Resolves the multi-input ambiguity: if several inputs are active, the encoder reports the one with **highest priority** (highest subscript, by convention). Adds a valid output $V = 1$ iff any input is active.

4-input priority encoder truth table (X = don't-care, allowing condensed listing):

D_0	D_1	D_2	D_3	x	y	V
0	0	0	0	X	X	0
1	0	0	0	0	0	1
X	1	0	0	0	1	1
X	X	1	0	1	0	1
X	X	X	1	1	1	1

Boolean output equations:

$$x = D_2 + D_3$$

$$y = D_3 + D_1 D_2'$$

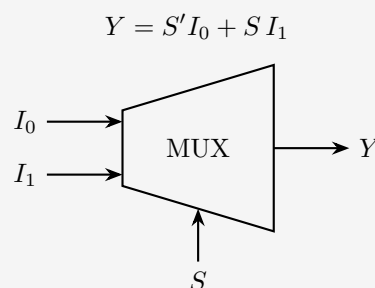
$$V = D_0 + D_1 + D_2 + D_3$$

★ **Don't-cares in truth tables** are useful in two distinct roles: in the *output* columns they license either value during minimisation; in the *input* columns they compress multiple rows into one (each X stands for both 0 and 1).

4.11 Multiplexers

A **multiplexer** (MUX, مَضَاعِف الإرسال) selects **one of 2^n inputs** and routes it to a single output. n **selection lines** pick which input.

2-to-1 MUX:



4-to-1 MUX function table:

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

Quadruple 2-to-1 MUX. Four parallel 2-to-1 MUXes share the same select S — used to choose between two 4-bit buses.

★ **A MUX = decoder + output OR.** The selection-decoding logic inside a 2^n -to-1 MUX is identical to an n -to- 2^n decoder; the only addition is one OR gate combining the data-gated minterms.

Boolean Function Implementation with a MUX

Naïve method: for a function of n variables, use a 2^n -to-1 MUX with all n inputs as selectors and the data inputs = truth-table outputs.

★ **Smarter method (saves a selector line):** Use a 2^{n-1} -to-1 MUX.

1. Choose $n-1$ variables as selectors (in order).
2. For the remaining variable z , examine each pair of truth-table rows that share the selector pattern:
 - Both 0 $\Rightarrow$ data input = 0.
 - Both 1 $\Rightarrow$ data input = 1.
 - $F = z$ on that pair $\Rightarrow$ data input = z .
 - $F = z'$ on that pair $\Rightarrow$ data input = z' .

Worked example. $F(x, y, z) = \Sigma(1, 2, 6, 7)$ with a 4-to-1 MUX, selectors $S_1=x$, $S_0=y$:

x	y	F when $z=0, z=1$	data input
0	0	0, 1	z
0	1	1, 0	z'
1	0	0, 0	0
1	1	1, 1	1

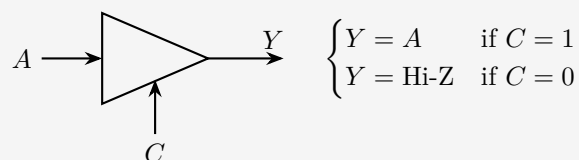
Result: 4-input MUX with data inputs $\{z, z', 0, 1\}$ implements F .

Extension. For 4 variables (A, B, C, D , $F = \Sigma(1, 3, 4, 11, 12, 13, 14, 15)$), use an 8-to-1 MUX with A, B, C on the selectors, and data inputs picked from $\{0, 1, D, D'\}$ row-pair by row-pair.

Three-State Buffers

A **three-state** (tri-state) gate has three output values:

- logic 0,
- logic 1, or
- **high-impedance (Hi-Z)** — output effectively disconnected.



Why Hi-Z matters: ordinary gate outputs cannot be wired together — competing 0 and 1 would short. Tri-state outputs *can* share a common “bus” wire as long as **at most one is enabled at any moment**.

MUX from tri-state buffers: feed each data input through its own tri-state buffer; tie all outputs to one wire; let a decoder enable exactly one buffer based on the select inputs. The result is a clean 2^n -to-1 multiplexer without internal OR gates.

Trap !!: Never enable two tri-state outputs onto the same bus

Doing so causes a logic conflict (and a real short-circuit current in CMOS) — the bus value is undefined and the chip may be damaged. Always make sure your enable logic is mutually exclusive.

Chapter 4 — Summary

- **Combinational $\neq$ Sequential.** No feedback, no memory $\Rightarrow$ combinational.
- **Analysis = circuit $\rightarrow$ function. Design = function $\rightarrow$ circuit.** Same skill, opposite direction.
- **Half adder:** $S = x \oplus y$, $C = xy$. **Full adder:** $S = x \oplus y \oplus z$, $C = \text{majority}(x, y, z)$.
- **Ripple carry adder:** simple but slow — $2n$ gate-levels of carry delay.
- **Carry-Lookahead** ($G_i = A_i B_i$, $P_i = A_i \oplus B_i$): all carries in 2 gate-levels, at the cost of wider gates.
- **Subtraction** = $A + B' + 1$. One XOR-per-bit and a mode line M turn an adder into a subtractor; $C_0 = M$.
- **Overflow (signed):** $V = C_{n-1} \oplus C_n$. **Carry-out C_n is the unsigned overflow flag.**
- **BCD adder:** 4-bit binary adder + correction (add 0110 if $K + Z_8 Z_4 + Z_8 Z_2$).
- **Multiplier:** $J \cdot K$ ANDs + $(J-1)$ K -bit adders $\rightarrow (J+K)$ -bit product.
- **Comparator:** use equality bits $x_i = A_i \odot B_i$; greater/less are MSB-first cascades.
- **Decoder:** $n \rightarrow 2^n$ unique outputs (one minterm each). Doubles as demultiplexer. Two enabled decoders = a bigger decoder.
- **Decoder + OR gates** implements any sum-of-minterms function.
- **Priority encoder:** resolves multi-input ambiguity by ranking inputs; valid bit V handles “no input”.
- **MUX:** 2^n -to-1 selector. n selection lines. Inside it is a decoder + OR.
- **MUX trick:** a function of n vars fits a 2^{n-1} -to-1 MUX, with data inputs $\in \{0, 1, z, z'\}$.
- **Tri-state buffer:** adds Hi-Z $\Rightarrow$ enables shared buses. Only one driver active at a time, ever.

تم تلخيص الوحدة الرابعة

Chapter 5 : Synchronous Sequential Logic

5.1 Introduction — Memory Enters the Picture

Every chapter so far built **combinational** circuits: outputs depend **only** and **immediately** on the present inputs. No memory, no history.

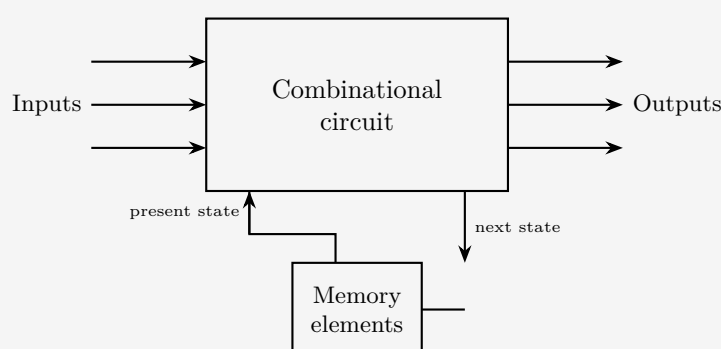
Cell phones, computers, cameras, navigation receivers — every modern electronic device needs to *store* and *retrieve* information. That ability comes from **sequential circuits** (دارات متتابعة): circuits whose outputs depend on the present inputs **and** on a stored internal state.

★ **One-line distinction:**

- Combinational: $\text{output} = f(\text{inputs})$.
- Sequential: $\text{output} = f(\text{inputs}, \text{state})$, $\text{next state} = g(\text{inputs}, \text{state})$.

5.2 Sequential Circuits — Block Diagram and Classification

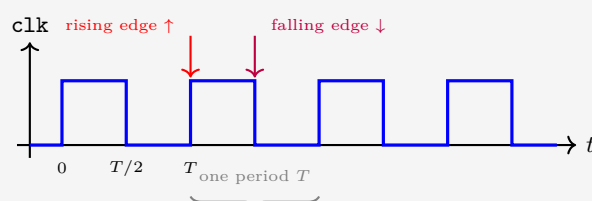
A sequential circuit = a combinational circuit + **storage elements** closed in a **feedback loop**.



Two flavors of sequential circuits (classified by signal timing):

- **Synchronous** (متزامن): state changes only at **discrete instants** dictated by a **clock** signal. Predictable, easy to design. *This is what we will study.*
- **Asynchronous** (غير متزامن): state changes *whenever* an input changes; storage is provided by raw gate propagation delays. Can become unstable. Out of scope for this course.

The clock signal (clk) is a periodic train of pulses generated by a clock generator. It marks the moments when the circuit is allowed to update its state.



Two kinds of triggers:

- **Rising edge** (positive edge, الحافة الصاعدة): clk goes $0 \rightarrow 1$.
- **Falling edge** (negative edge, الحافة الهابطة): clk goes $1 \rightarrow 0$.

★ **The key idea of synchronicity:** we want **one state change per pulse**. The storage element must respond to a **moment** (an edge), not to a **period** (a level). Otherwise, while the clock is still high, the new state could feed back through the combinational circuit and trigger *another* change, and another...

5.3 Storage Elements: Latches

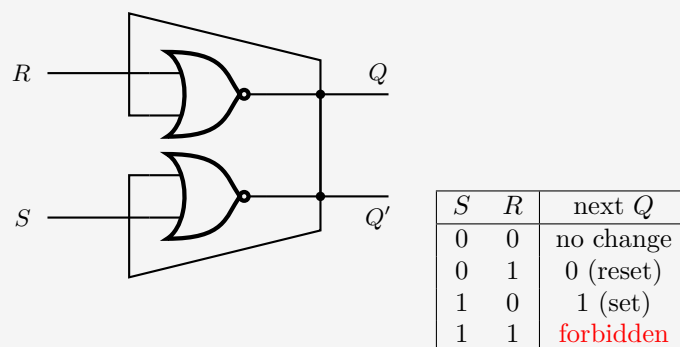
A **storage element** holds its binary value indefinitely (as long as power is on) until an input tells it to change.

- **Latch** (مزلاج): **level-sensitive** — responds to the *level* of its control input (high or low).
- **Flip-flop** (قلاب): **edge-sensitive** — responds only at the *edge* (transition) of its control input.

Latches are the building blocks; flip-flops are built from latches. We start with latches.

SR Latch with NOR gates (active-HIGH)

Two cross-coupled NOR gates, two inputs S (set) and R (reset).



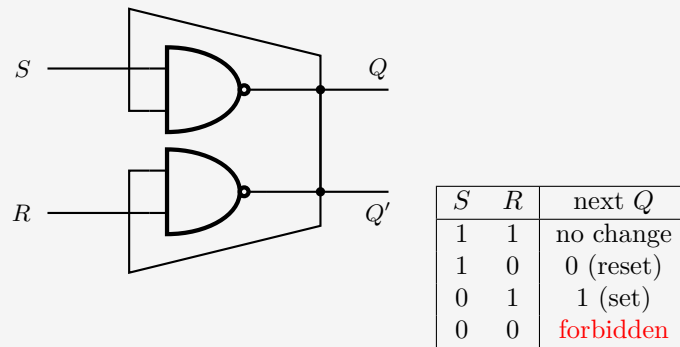
Reading the function table. NOR latch is **active-HIGH**: rest state is $S = R = 0$. A momentary $S = 1$ sets the latch ($Q \rightarrow 1$); a momentary $R = 1$ resets it ($Q \rightarrow 0$). After the active input returns to 0, the latch *remembers*.

Trap !!: The forbidden input

Setting $S = R = 1$ simultaneously forces **both** outputs to 0, violating the rule $Q' = \overline{Q}$. If both inputs then drop to 0 *at the same time*, the latch enters an undefined / metastable state — you cannot predict whether it will settle to set or reset. **Never apply** $S = R = 1$.

SR Latch with NAND gates (active-LOW)

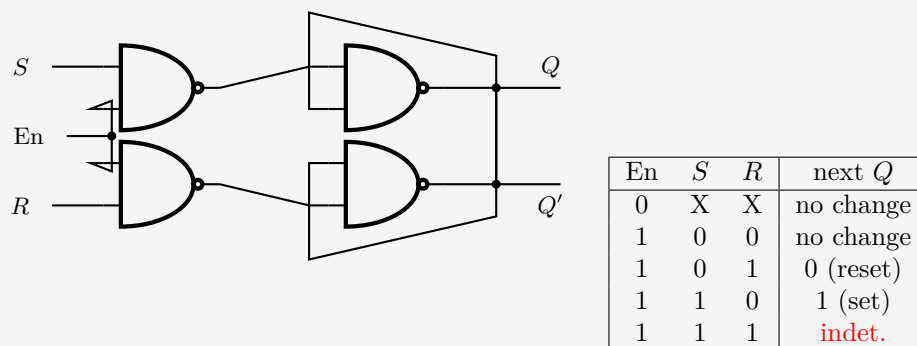
Identical idea, but built from cross-coupled NANDs — the inputs are **active-LOW** (everything inverted).



Rest state is $S = R = 1$. A momentary $S = 0$ sets, momentary $R = 0$ resets. **Forbidden input** is $S = R = 0$.

SR Latch with Enable (Gated SR Latch)

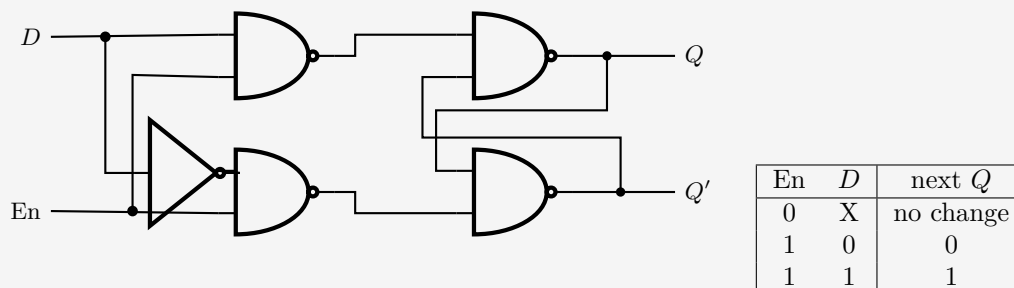
Add a control input En that decides *when* the latch is allowed to look at S, R . Two extra NANDs gate the inputs.



★ **Mental model:** the Enable input is a **switch**. When $En = 0$, the latch is *frozen* regardless of S, R . When $En = 1$, S and R are passed through to the inner SR latch and behave as before.

D Latch (Transparent Latch)

To kill the forbidden-input problem permanently: tie $S = D$ and $R = D'$ so that they can *never* be equal at the same time. The result is the **D latch**.

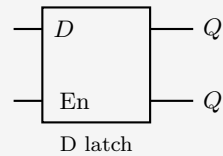
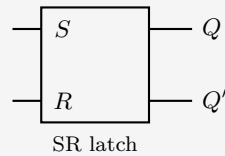


Behavior:

- $En = 1$: Q **tracks** D continuously. The latch is "transparent" — a window from D straight to Q .

- $En = 0$: Q is **latched** (frozen) at whatever value D had at the moment of closing.

Graphic symbols:



5.4 Storage Elements: Flip-Flops

Why Latches Are Not Enough for Synchronous Circuits

Picture a sequential circuit using D *latches* clocked by En . The moment En goes high, the new Q values feed back through the combinational circuit. If that fed-back signal arrives at the latch input *while* En is **still** high, the latch updates again. And again. As long as En stays high, the circuit can keep changing unpredictably.

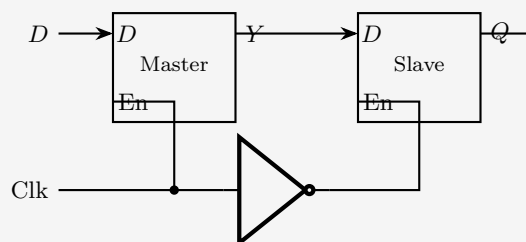
★ **The fix:** the storage element must be sensitive to a *moment in time* (an **edge**), not a *stretch of time* (a level). That moment is so brief that the new Q has no chance to propagate back and re-trigger anything before the moment is over.

Two standard ways to convert a latch into an edge-triggered flip-flop:

- **Master-slave construction:** two latches in series, clocked on opposite phases.
- **Three-SR-latch construction:** responds only on one edge of the clock.

Master-Slave D Flip-Flop

Two D latches in series, the second clock-inverted. The first ("master") captures D while $Clk = 1$; the second ("slave") copies the master while $Clk = 0$. The output Q only updates on the **negative edge** of the clock.



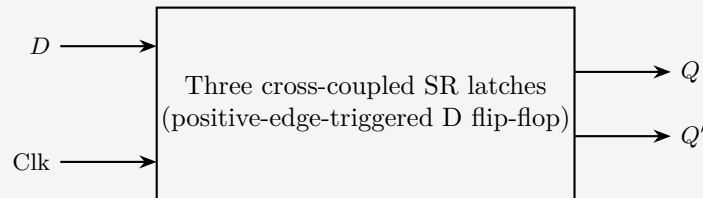
Step by step:

- $Clk = 1$: master is transparent ($Y = D$), slave is frozen (Q doesn't change).
- $Clk = 0$: master is frozen (holds last D), slave is transparent ($Q = Y$).
- On the $1 \rightarrow 0$ edge: master stops capturing, slave copies the captured value to Q .

Adding an *extra* inverter on Clk flips this to a **positive-edge**-triggered flip-flop.

Positive-Edge D Flip-Flop with Three SR Latches

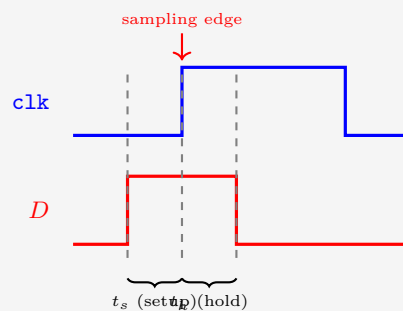
Another popular construction — and the one drawn in your handwritten notes. Three SR latches; the first two react to D and Clk , the third holds the output. On the rising edge of Clk , the captured value of D is gated through to Q ; once it's through, the input path locks itself so further changes in D have no effect until the next rising edge.



Properties (regardless of construction): on the positive (or negative) edge of Clk , the flip-flop samples D and transfers it to Q . *Changes in D at any other time are ignored.*

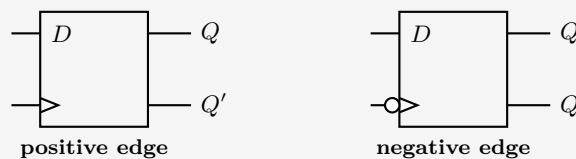
Timing parameters (real hardware constraints):

- **Setup time t_s :** how long D must be *stable before* the clock edge.
- **Hold time t_h :** how long D must stay stable *after* the clock edge.
- **Propagation delay t_{pd} :** from the clock edge until the new Q stabilizes.



Violating t_s or t_h can drive the flip-flop into a metastable state with no well-defined output.

Graphic symbols (positive vs negative edge):



The little wedge $\triangleright$ inside the box marks an edge-sensitive (dynamic) clock input. A bubble outside means *negative-edge*.

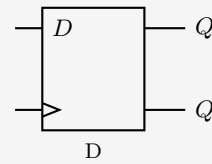
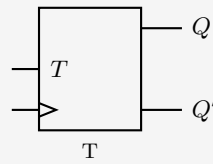
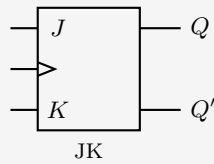
JK and T Flip-Flops — Two Cousins of D

A flip-flop can do three things on a clock edge: **set** (force to 1), **reset** (force to 0), or **toggle** (complement). The D flip-flop only sets/resets. The JK does all three; the T only toggles or holds.

- **JK Flip-flop:** built as a D flip-flop with $D = JQ' + K'Q$. The J input sets, K resets, and

$J = K = 1$ toggles.

- **T Flip-flop:** built as a D flip-flop with $D = T \oplus Q$. When $T = 1$ the output toggles, when $T = 0$ it holds.



T flip-flop from a JK: short J and K together, call the common line T .

T flip-flop from a D: feed $D = T \oplus Q$ back through an XOR.

Characteristic Tables — The Big Cheat Sheet

Each flip-flop's behaviour is summarised by its **characteristic table**: “given the inputs and the present state $Q(t)$, what is the next state $Q(t+1)$?”

JK Flip-Flop		
J	K	$Q(t+1)$
0	0	$Q(t)$ (no change)
0	1	0 (reset)
1	0	1 (set)
1	1	$Q'(t)$ (toggle)

D Flip-Flop	
D	$Q(t+1)$
0	0 (reset)
1	1 (set)

T Flip-Flop	
T	$Q(t+1)$
0	$Q(t)$
1	$Q'(t)$

★ **Characteristic equations** (algebraic form of the tables — memorise these):

$$\text{D: } Q(t+1) = D$$

$$\text{JK: } Q(t+1) = JQ' + K'Q$$

$$\text{T: } Q(t+1) = T \oplus Q = TQ' + T'Q$$

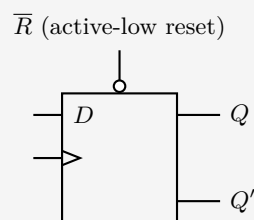
Direct (Asynchronous) Inputs: Preset & Clear

Some flip-flops have extra inputs that override the clock:

- **Preset (direct set):** forces $Q \rightarrow 1$ immediately, ignoring Clk and D .
- **Clear (direct reset):** forces $Q \rightarrow 0$ immediately.

Why? When power is first turned on, every flip-flop has a random initial state. Asserting **reset** once at startup brings the whole system into a known starting state.

Graphic symbol with active-LOW direct reset:



5.5 Analysis of Clocked Sequential Circuits

Analysis answers: “given a circuit, what does it *do*?” The output is one of (a) a **state table**, (b) a **state diagram**, or (c) a set of Boolean equations.

The toolbox:

- **Flip-flop input equations** (a.k.a. excitation equations): Boolean expressions for $D_A, J_A, K_A, T_A, \dots$ in terms of present states and inputs.
- **State equations** (transition equations): give the next-state value $A(t+1)$ as a Boolean function of present state and inputs. For D flip-flops, the state equation *equals* the input equation: $A(t+1) = D_A$.
- **Output equations:** give external outputs as Boolean functions of present state (and possibly inputs).

Pipeline:

Circuit diagram $\longrightarrow$ Equations $\longrightarrow$ State table $\longrightarrow$ State diagram

Worked Example: Analysis with D Flip-Flops

Given: a circuit with two D flip-flops A, B , one input x , one output y , described by

$$D_A = Ax + Bx$$

$$D_B = A'x$$

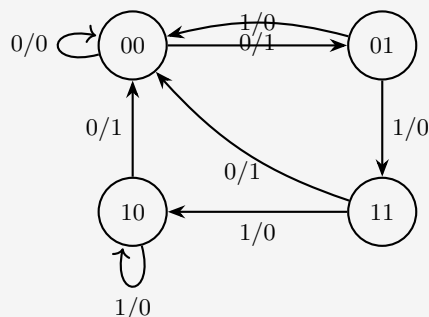
$$y = (A + B)x'$$

Step 1 — State equations. Since D-FF's: $A(t+1) = D_A$, $B(t+1) = D_B$.

Step 2 — State table. List all combinations of present state (A, B) and input x (so $2^3 = 8$ rows). Plug into the equations:

Present state		Input	Next state		Output
A	B	x	A	B	y
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	1
0	1	1	1	1	0
1	0	0	0	0	1
1	0	1	1	0	0
1	1	0	0	0	1
1	1	1	1	0	0

Step 3 — State diagram. One circle per state (4 states for 2 flip-flops). Each directed arrow is labelled x/y .



What does it do? Starting from state 00, while the input stays at 1 the output stays at 0. The first 0 in a stream of 1's produces an output of 1 and snaps the machine back to 00. **This is a zero-detector in a stream of 1's.**

Analysis with JK Flip-Flops

Same plan, but now the next-state value is **not** equal to the input equation: it comes from the JK characteristic table (or equation) $Q(t+1) = JQ' + K'Q$.

Recipe:

1. Write the input equations for $J_A, K_A, J_B, K_B, \dots$
2. For each row of present-state/input, compute the four input values.
3. Look up $Q(t+1)$ in the JK characteristic table — **or** plug into the state equation $A(t+1) = J_A A' + K'_A A$.

Example. JK FFs with $J_A = B$, $K_A = Bx'$, $J_B = x'$, $K_B = A \oplus x$:

Present		Input	Next		FF inputs			
A	B		A	B	J_A	K_A	J_B	K_B
0	0	0	0	1	0	0	1	0
0	0	1	0	0	0	0	0	1
0	1	0	1	1	1	1	1	0
0	1	1	1	0	1	0	0	1
1	0	0	1	1	0	0	1	1
1	0	1	1	0	0	0	0	0
1	1	0	0	0	1	1	1	1
1	1	1	1	1	1	0	0	0

How a single row is filled. Take the row $ABx = 110$. Then $J_A = B = 1$, $K_A = Bx' = 1 \cdot 1 = 1$, so $JK = 11 \Rightarrow \text{toggle}$, so $A : 1 \rightarrow 0$. Similarly $J_B = x' = 1$, $K_B = A \oplus x = 1$, so $B : 1 \rightarrow 0$.

Analysis with T Flip-Flops — A Binary Counter

Same idea, characteristic equation now $Q(t+1) = T \oplus Q$.

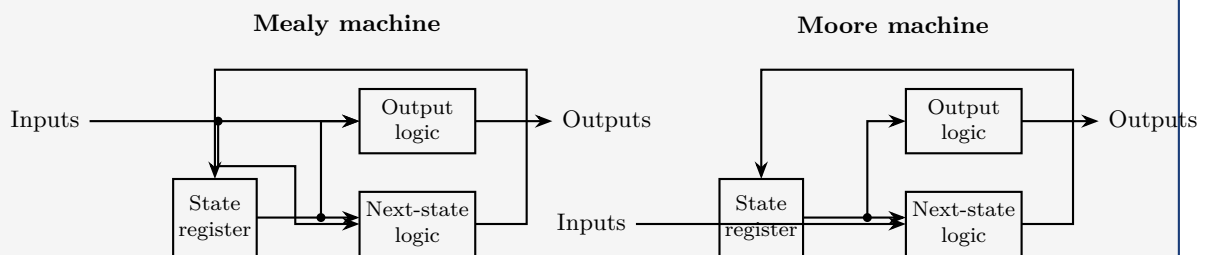
Example. Two T FFs with $T_A = Bx$, $T_B = x$, output $y = AB$.

Present		Input	Next		Output
A	B	x	A	B	y
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	1	0
0	1	1	1	0	0
1	0	0	1	0	0
1	0	1	1	1	0
1	1	0	1	1	1
1	1	1	0	0	1

Read it as: when $x = 1$, the machine counts $00 \rightarrow 01 \rightarrow 10 \rightarrow 11 \rightarrow 00 \rightarrow \dots$. **This is a 2-bit binary counter.** y flags when the count reaches 11. When $x = 0$ the count is paused.

Mealy vs Moore Machines

Two flavors of finite-state machine (**FSM**), distinguished by *where* outputs come from:



- **Mealy:** output = $f(\text{state}, \text{input})$. Output can change *between clock edges* if input changes. Labels on state-diagram arrows: input/output.
- **Moore:** output = $f(\text{state})$ only. Output is synchronous with the clock. Output labels go *inside* each state circle: state/output, and arrows carry only the input.

★ **Quick test:** look at a state diagram.

- Output labels on the *arrows* $\Rightarrow$ Mealy.
- Output labels *inside the circles* $\Rightarrow$ Moore.

Trap !!: Mealy outputs can glitch

Mealy outputs may show momentary false values when inputs change asynchronously to the clock. If you need a clean, clock-synchronous output, prefer Moore (or register the Mealy output through one extra flip-flop).

5.7 State Reduction and State Assignment

State Reduction — Removing Redundant States

Two sequential circuits are **equivalent** (in I/O behaviour) if they produce identical output sequences for every input sequence. Fewer states $\Rightarrow$ usually fewer flip-flops $\Rightarrow$ usually cheaper circuit.

Equivalence rule: *Two states are equivalent if, for every input, they produce the same output and go to the same (or equivalent) next state.*

Reduction algorithm:

1. Build the state table.
2. Scan rows: find two rows with identical *next-state* and identical *output* columns for *every* input.
3. Delete one row; replace its name everywhere by the surviving row's name.
4. Repeat until no more merges are possible.

Worked example. Starting from a 7-state machine (a, b, c, d, e, f, g):

Present state	Next state		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
a	a	b	0	0
b	c	d	0	0
c	a	d	0	0
d	e	f	0	1
e	a	f	0	1
f	g	f	0	1
g	a	f	0	1

Rows e and g are identical $\Rightarrow$ merge $g \rightarrow e$. After replacing g everywhere:

Now rows d and f become identical $\Rightarrow$ merge $f \rightarrow d$.

Final reduced table (5 states):

Present	Next state		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
a	a	b	0	0
b	c	d	0	0
c	a	d	0	0
d	e	d	0	1
e	a	d	0	1

From 7 states to 5 states. Same input/output behaviour, possibly less hardware.

Trap !!: Fewer states does **not** always mean fewer flip-flops. 7 states or 5 states both need $\lceil \log_2 \rceil = 3$ flip-flops in standard binary encoding. The win comes from the *additional don't-care states* that simplify the combinational logic.

State Assignment — Putting Bits on States

To realize a state machine in hardware, every state needs a unique binary code. For m states you need n bits with $2^n \geq m$.

Three common encoding styles:

State	Binary	Gray code	One-hot
a	000	000	00001
b	001	001	00010
c	010	011	00100
d	011	010	01000
e	100	110	10000

- **Binary (sequential):** $\lceil \log_2 m \rceil$ flip-flops. Most compact. Default choice.
- **Gray code:** only one bit flips between consecutive states; can simplify K-map terms when state transitions are mostly sequential.
- **One-hot:** m flip-flops, exactly one bit is 1 at any time. Wastes flip-flops but the next-state / output logic is usually simpler and faster — a great fit for register-rich **FPGAs**.

★ **Unused codes become don't-cares.** If you have 5 states encoded in 3 bits, codes 101, 110, 111 are unreachable. Treat them as “X” in K-maps for free simplification.

5.8 Design Procedure — From Spec to Schematic

The whole point of this chapter. Given a word description, build the circuit.

Seven recommended steps:

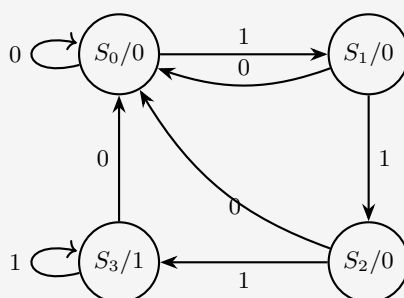
1. From the word description, draw the **state diagram**.
2. Reduce the number of states if needed.
3. Assign binary codes to states.
4. Obtain the binary-coded **state table**.
5. Choose the type of flip-flops (D, JK, or T).
6. Derive simplified **flip-flop input equations** and **output equations**.
7. Draw the **logic diagram**.

Steps 4–7 are mechanical and automatable; step 1 is where the human judgment lives.

Synthesis Using D Flip-Flops — Sequence Detector

Spec. Design a circuit that outputs $y = 1$ whenever it detects **three or more consecutive 1's** in a serial input stream x .

Step 1 — State diagram. Four states tracking “how many 1's in a row have I seen?”: S_0, S_1, S_2, S_3 . A 0 always returns to S_0 . (Moore machine: output is the state itself.)



Step 2 & 3 — Reduce / assign. 4 states $\Rightarrow$ no reduction possible. Use 2-bit binary assignment

$S_0 = 00$, $S_1 = 01$, $S_2 = 10$, $S_3 = 11$ on FFs A, B .

Step 4 — State table:

Present		Input	Next		Output
A	B	x	A	B	y
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	0
0	1	1	1	0	0
1	0	0	0	0	0
1	0	1	1	1	0
1	1	0	0	0	1
1	1	1	1	1	1

Steps 5–6 — D flip-flops, input equations. For D-FFs, the input equation **equals** the next-state expression. Read minterms directly off the table:

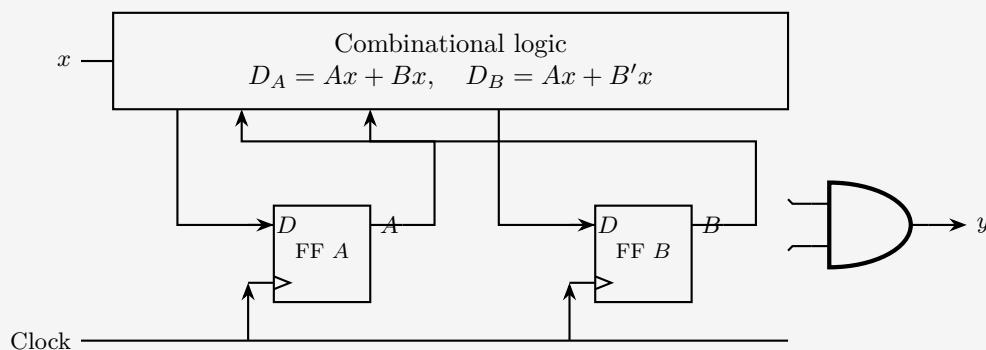
$$D_A = A(t+1) = \Sigma m(3, 5, 7) = Ax + Bx$$

$$D_B = B(t+1) = \Sigma m(1, 5, 7) = Ax + B'x$$

$$y = \Sigma m(6, 7) = AB$$

(K-map simplifications — omitted but trivial.)

Step 7 — Schematic.



Excitation Tables — The Other Direction

For D flip-flops, the input equation = the next state, so design is easy. For JK and T, we need the **excitation table**: “given the desired transition $Q(t) \rightarrow Q(t+1)$, what should the inputs be?”

JK Flip-Flop			
$Q(t)$	$Q(t+1)$	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

T Flip-Flop		
$Q(t)$	$Q(t+1)$	T
0	0	0
0	1	1
1	0	1
1	1	0

D Flip-Flop		
$Q(t)$	$Q(t+1)$	D
0	0	0
0	1	1
1	0	0
1	1	1

★ **Why the X's matter:** the don't-cares in the JK table mean lots of K-map flexibility → usually simpler combinational logic than D for manual design.

Mnemonic for T: $T = 1$ iff the state must *change*, i.e. $T = Q(t) \oplus Q(t+1)$.

Mnemonic for JK:

- $0 \rightarrow 0$: $J = 0$ (don't set), $K = X$.
- $0 \rightarrow 1$: $J = 1$ (must set), $K = X$.
- $1 \rightarrow 0$: $J = X$, $K = 1$ (must reset).
- $1 \rightarrow 1$: $J = X$, $K = 0$ (don't reset).

Synthesis Using JK Flip-Flops

Identical procedure, but step 6 uses the JK excitation table to fill in J and K columns row by row.

Example state table augmented with JK inputs:

Present		Input	Next		Required FF inputs			
A	B	x	A	B	J_A	K_A	J_B	K_B
0	0	0	0	0	0	X	0	X
0	0	1	0	1	0	X	1	X
0	1	0	1	0	1	X	X	1
0	1	1	0	1	0	X	X	0
1	0	0	1	0	X	0	0	X
1	0	1	1	1	X	0	1	X
1	1	0	1	1	X	0	X	0
1	1	1	0	0	X	1	X	1

Each (J_A, K_A) pair is read row by row from the JK excitation table given the $A \rightarrow A'$ transition. Then K-map each column to get the input equations:

$$J_A = Bx', \quad K_A = Bx$$

$$J_B = x, \quad K_B = (A \oplus x)'$$

The many X's in the columns mean lots of room to grow K-map groups $\Rightarrow$ small, simple equations.

Synthesis Using T Flip-Flops — 3-bit Binary Counter

T flip-flops shine at **counters**: since $T = 1$ flips a bit, “flip when needed” maps naturally to counting.

Spec: 3-bit binary counter cycling $000 \rightarrow 001 \rightarrow 010 \rightarrow \dots \rightarrow 111 \rightarrow 000 \rightarrow \dots$

State table with T inputs (filled from excitation table $T = Q(t) \oplus Q(t+1)$):

Present			Next			FF inputs		
A_2	A_1	A_0	A_2	A_1	A_0	T_{A_2}	T_{A_1}	T_{A_0}
0	0	0	0	0	1	0	0	1
0	0	1	0	1	0	0	1	1
0	1	0	0	1	1	0	0	1
0	1	1	1	0	0	1	1	1
1	0	0	1	0	1	0	0	1
1	0	1	1	1	0	0	1	1
1	1	0	1	1	1	0	0	1
1	1	1	0	0	0	1	1	1

Pattern! $T_{A_0} = 1$ always (LSB toggles every clock). $T_{A_1} = 1$ when $A_0 = 1$. $T_{A_2} = 1$ when

$A_0A_1 = 1$. So:

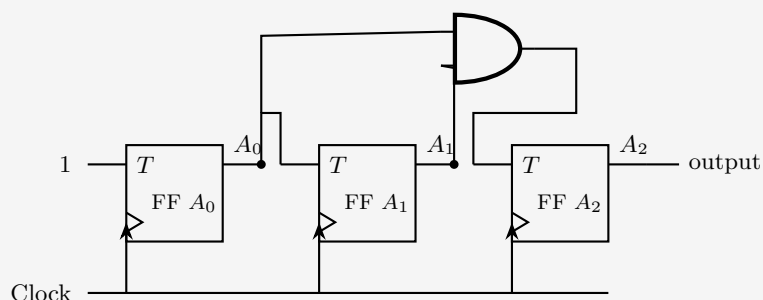
$$T_{A_0} = 1$$

$$T_{A_1} = A_0$$

$$T_{A_2} = A_0A_1$$

★ **General rule for a binary counter:** bit A_i toggles whenever all lower bits are 1, i.e. $T_{A_i} = A_0A_1 \cdots A_{i-1}$.

Schematic:



Chapter 5 — Summary

- **Combinational $\neq$ Sequential.** Sequential = combinational + feedback through storage elements (memory).
- **Synchronous** circuits update only at **clock edges**. **Asynchronous** ones can change anytime — and can become unstable. We only study synchronous.
- **Latch (level-sensitive) $\neq$ Flip-flop (edge-sensitive).** Latches are the building blocks; flip-flops are what we actually use in clocked circuits.
- **SR latch:** cross-coupled NOR (active-HIGH) or NAND (active-LOW). Forbidden input $\rightarrow$ indeterminate state. NEVER allowed.
- **D latch** (transparent): $D = S, D' = R$. $En = 1 \Rightarrow Q$ follows D ; $En = 0 \Rightarrow Q$ frozen.
- **D flip-flop** = two D latches (master-slave) OR three SR latches. Samples D *only on the clock edge*.
- **Setup/Hold/Propagation** times are real hardware constraints; violating them produces metastability.
- **Characteristic equations** (next-state formulas):
 - D: $Q(t+1) = D$
 - JK: $Q(t+1) = JQ' + K'Q$
 - T: $Q(t+1) = T \oplus Q$
- **Direct inputs** (Preset / Clear) override the clock — used for initialisation at power-on.
- **Analysis pipeline:** Circuit $\rightarrow$ Equations $\rightarrow$ State table $\rightarrow$ State diagram.
- **Mealy** machine: output depends on **state + input**, label on arrows. **Moore:** output depends only on **state**, label inside circles.
- **State reduction:** merge states that produce the same outputs and go to the same (or equivalent) next states for every input.
- **State assignment:** binary (compact), Gray (one-bit-change), one-hot (one FF per state, simpler logic for FPGAs).

- **Design = the reverse of analysis:** word spec $\rightarrow$ state diagram $\rightarrow$ table $\rightarrow$ pick FF type $\rightarrow$ K-map the input equations $\rightarrow$ draw.
- **Excitation table** = “what inputs achieve a desired transition?” Don’t-cares in the JK excitation table give it a manual-design edge over D.
- **T flip-flops are perfect for counters** because $T_{A_i} = A_0 A_1 \cdots A_{i-1}$ (bit i toggles when all lower bits are 1).

تم تلخيص الوحدة الخامسة

تم بحمد الله